

IOOP GROUP ASSIGNMENT

INTAKE CODE: APU/APD1F2509CS(AI)

MODULE CODE: CT044-3-1-IOOP-L-4

MODULE NAME: INTRODUCTION TO OBJECT ORIENTED PROGRAMMING

LAB GROUP: LECTURE 4 (LAB 10 & LAB 11)

TITLE: GR8FOOD MANAGEMENT SYSTEM

LECTURER NAME: DR. KULOTHUNKAN A/L PALASUNDRAM

SUBMISSION DATE: 30TH MAY 2026

Name	TP Number
Darren Matthew Tanoyo	TP086059
Duan Yuqing	TP114820
Farah Zakiya	TP092149
Omar Ahmed Ramy Mohammed Elsayed Hassan Mansour	TP078735

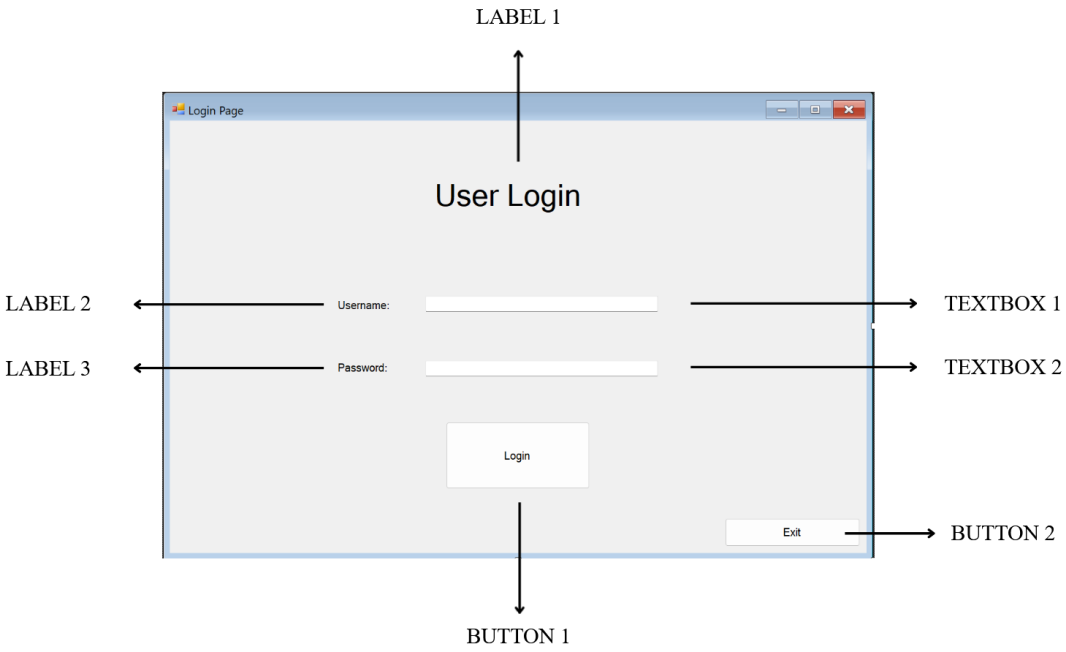
0. Table of contents

0. Table of contents 1
1. User Storyboard 2
1.1 Login page 3
1.2 Admin Pages 3
1.3 Customer Pages 4
1.4 Manager Pages 4
1.5 Chef Pages 4
1.6 Update Profile Page 4
2. Use-Case Diagram 4
3. Class Diagram 4
3.1 Admin 4
3.1.1 Admin Class Diagram 4
3.1.2 DBHelper Class Methods 4
3.1.3 Admin Class Methods 4
3.1.4 Chef Class Methods 4
3.1.5 Menu Class Methods 4
3.1.6 Order Class Methods 4
3.1.7 Customer Class Methods 4
3.1.8 Manager Class Methods 4
3.2 Customer 4
3.2.1 Customer Class Diagram 4
3.2.2 Customer Class 4
3.2.3 MenuItem 4
3.2.4 Order Details 4
3.2.5 Customer Feedback 4
3.2.6 DBHelper 4
3.3 Manager 4
3.3.1 Class Diagram 4
3.3.2 Feedback class 4
3.3.3 Manager class 4
3.3.4 ManagerWalletReportRepository class 4
3.3.5 DBHelper 4
3.4 Chef 4

3.4.1 Class Diagram....4
3.4.2 DBHelper class....4
3.4.3 Order class....4
3.4.4 Order Details class....4
4. Test Plan and Test Cases....4
5. Conclusion....4
6. References....4
7. Workload Matrix....4

1. User Storyboard

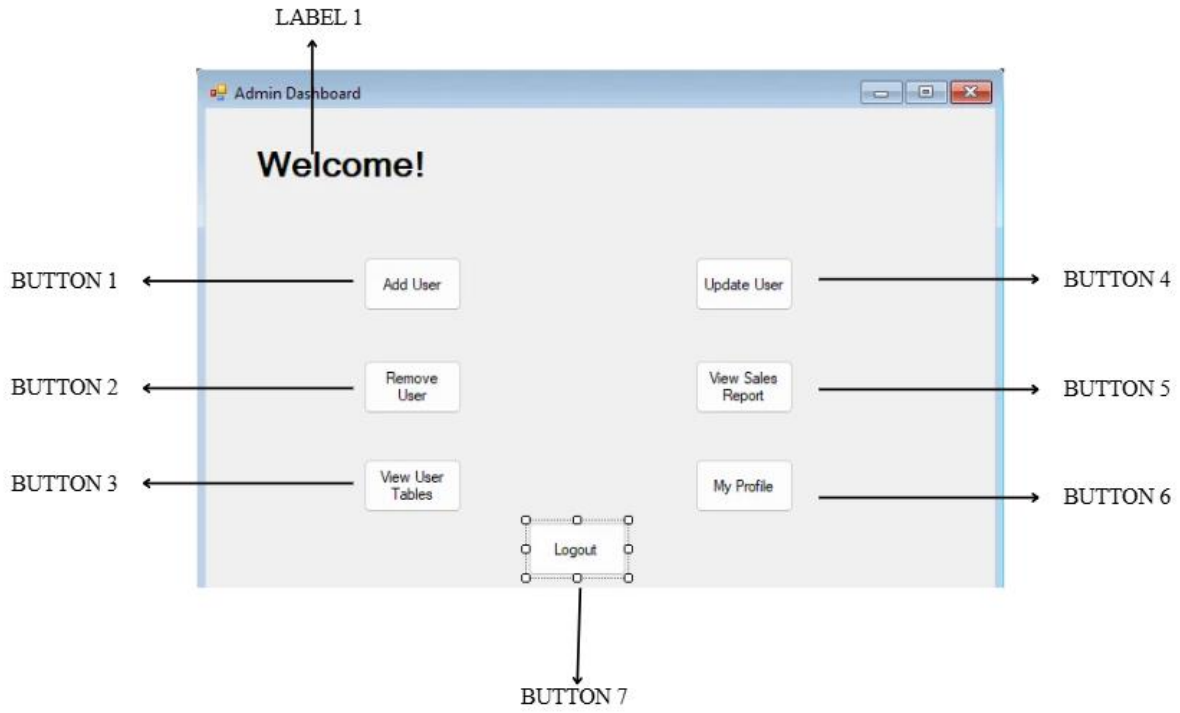
1.1 Login page



Control	Control Name	Description
Label 1	lblLogin	Display the Login page
Label 2	lblUsername	Display the related controls
Label 3	lblPassword	
Textbox 1	txtboxUsername	User input for username
Textbox 2	txtboxPassword	User input for password
Button 1	btnLogin	To login into the system
Button 2	btnExit	Exit

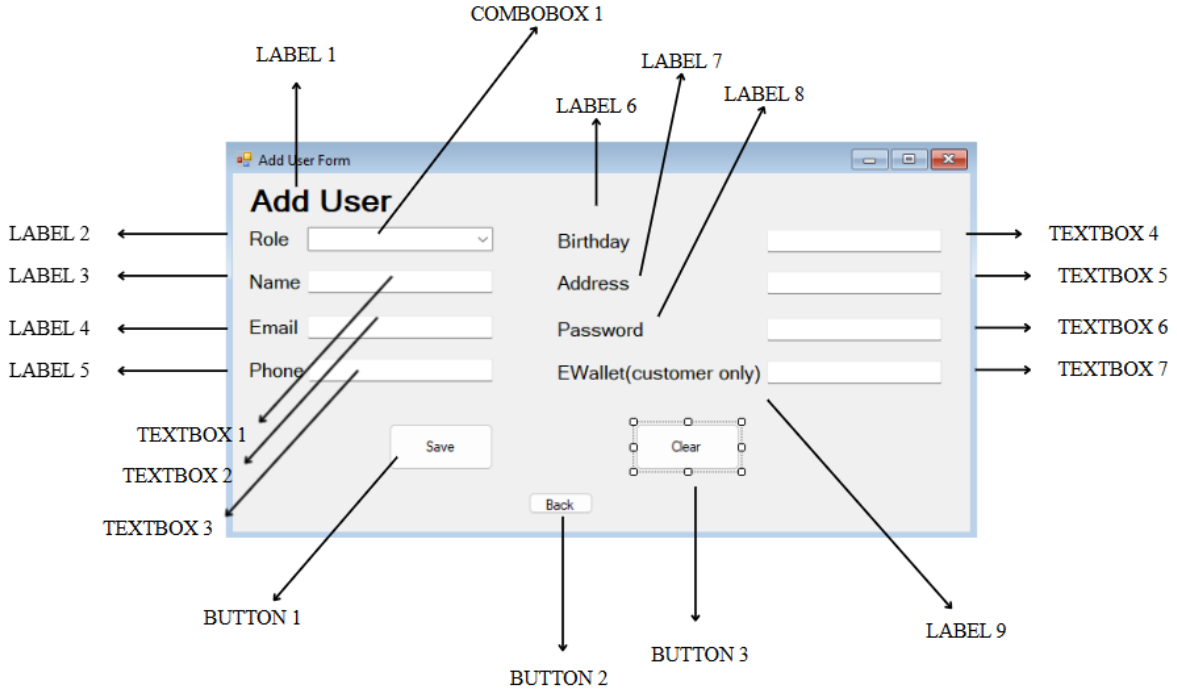
1.2 Admin Pages

1.2.1 Admin Dashboard



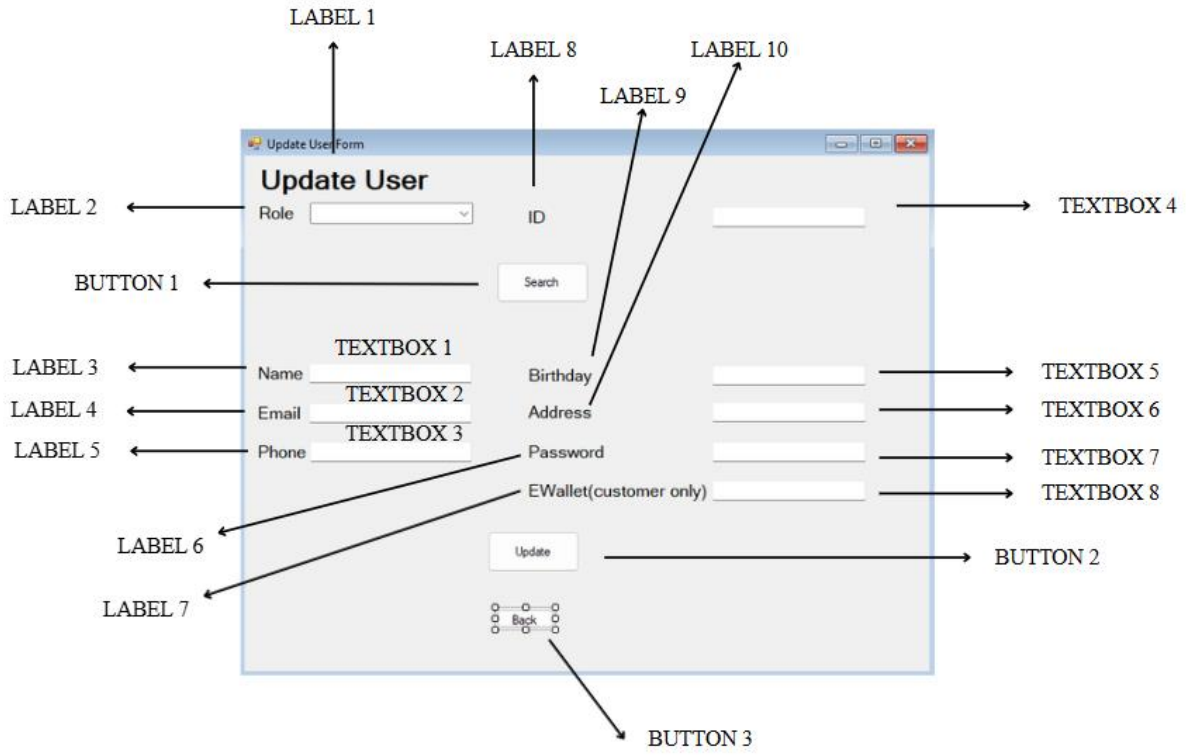
Control	Control Name	Description
Label 1	labelWelcome	Displays personalized welcome message
Button 1	btnAddUserForm	Opens the AddUserForm
Button 2	btnRemoveUserForm	Opens the RemoveUserForm
Button 3	btnViewTables	Opens the ViewTablesForm
Button 4	BtnUpdateUserForm	Opens the UpdateUserForm
Button 5	btnViewSalesReportForm	Opens the ViewSalesReportForm
Button 6	btnProfile	Opens the UpdateProfileForm
Button 7	btnLogout	Logs out and returns to the LoginPage

1.2.2 Add User Form



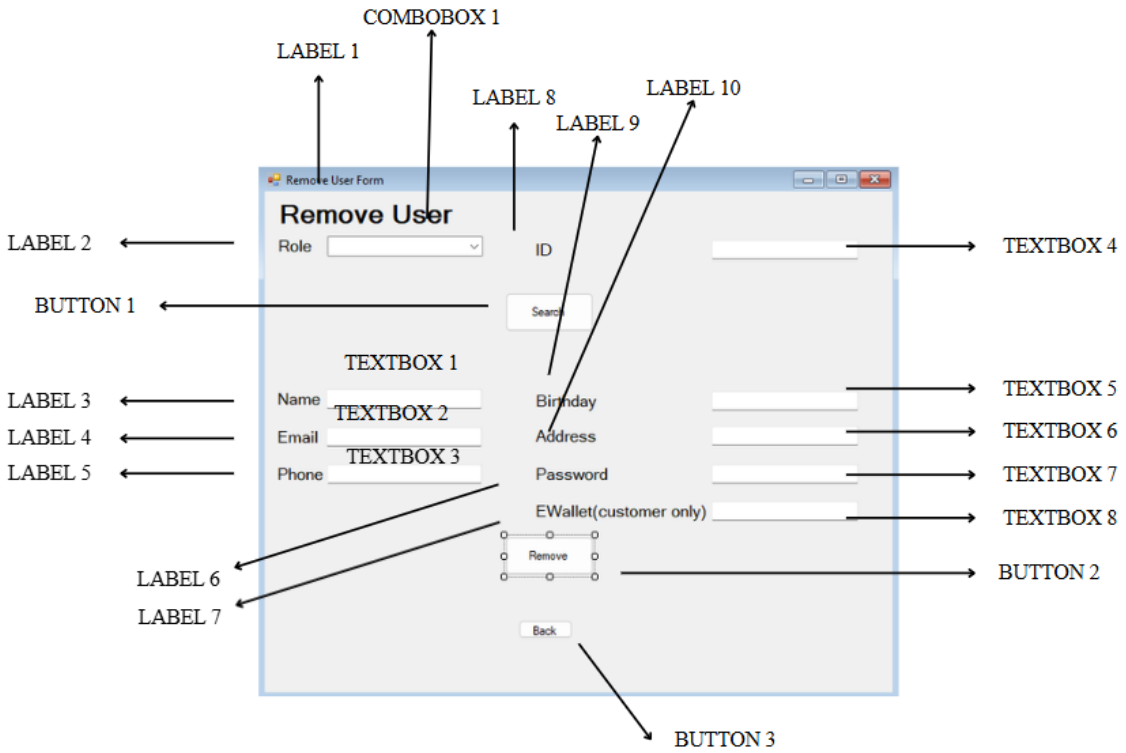
Control	Control Name	Description
Label 1	label1	Displays the page title "Add User"
Label 2	label2	Labels the role dropdown
Label 3	label3	Labels the name field
Label 4	label4	Labels the email field
Label 5	label5	Labels the phone field
Label 6	label6	Labels the birthday field
Label 7	label7	Labels the address field
Label 8	label8	Labels the password field
Label 9	label9	Labels the e-wallet field
ComboBox 1	cboRole	Dropdown to select user role (Admin, Manager, Chef, Customer)
TextBox 1	txtName	Input field for user's name
TextBox 2	txtEmail	Input field for user's email address
TextBox 3	txtPhone	Input field for user's phone number
TextBox 4	txtBirthday	Input field for birthday in YYYY-MM-DD format
TextBox 5	txtAddress	Input field for user's address
TextBox 6	txtPassword	Input field for user's password
TextBox 7	txtEWallet	Input field for e-wallet balance (enabled for Customer only)
Button 1	btnSave	Validates and saves the new user to the database
Button 2	btnBack	Returns to the Admin Dashboard
Button 3	btnClear	Clears all input fields

1.2.3 Update User Form



Control	Control Name	Description
Label 1	label1	Displays the page title "Update User"
Label 2	label2	Labels the role dropdown
Label 3	label3	Labels the name field
Label 4	label4	Labels the email field
Label 5	label5	Labels the phone field
Label 6	label8	Labels the password field
Label 7	label9	Labels the e-wallet field
Label 8	label10	Labels the search ID field
Label 9	label6	Labels the birthday field
Label 10	label7	Labels the address field
ComboBox 1	cboRole	Dropdown to select user role (Admin, Manager, Chef, Customer)
TextBox 1	txtName	Input field for user's name
TextBox 2	txtEmail	Input field for user's email address
TextBox 3	txtPhone	Input field for user's phone number
TextBox 4	txtSearchID	Input field for the target user's ID to search
TextBox 5	txtBirthday	Input field for birthday in YYYY-MM-DD format
TextBox 6	txtAddress	Input field for user's address
TextBox 7	txtPassword	Input field for user's password
TextBox 8	txtEWallet	Input field for e-wallet balance (enabled for Customer only)
Button 1	btnSearch	Searches the database for the specified ID and role
Button 2	btnUpdate	Validates and updates the user record in the database
Button 3	btnBack	Returns to the Admin Dashboard

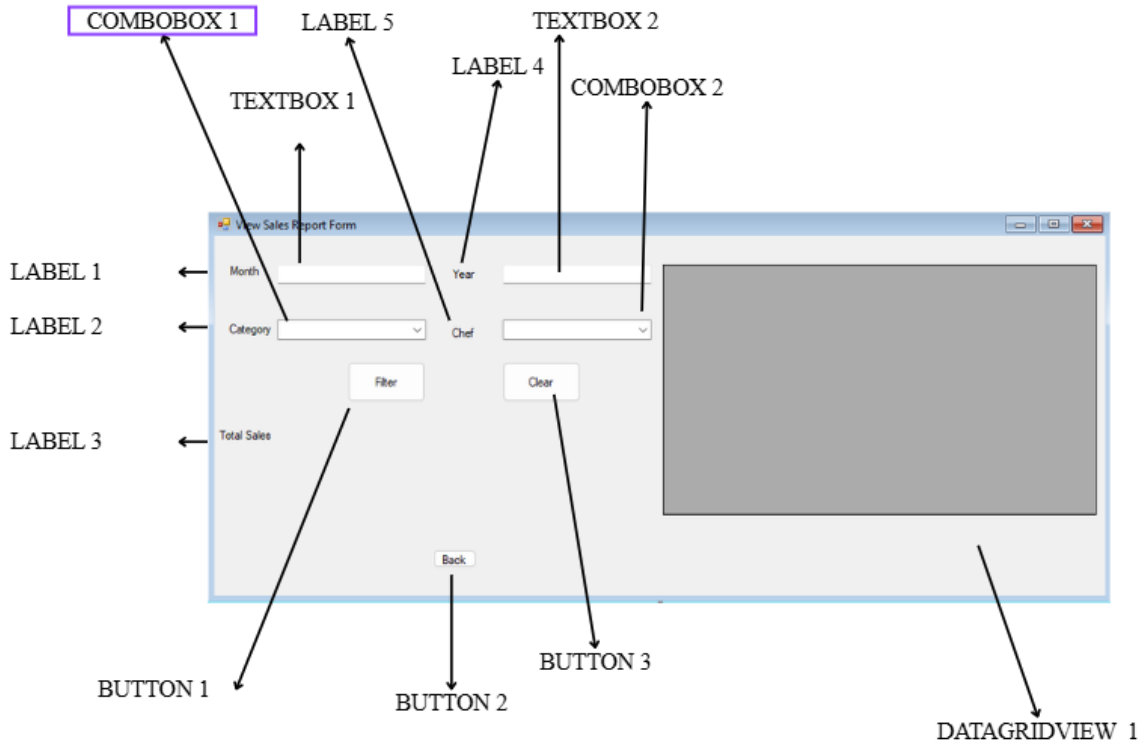
1.2.4 Remove User Form



Control	Control Name	Description
Label 1	label1	Displays the page title "Remove User"
Label 2	label2	Labels the role dropdown
Label 3	label3	Labels the name field
Label 4	label4	Labels the email field
Label 5	label5	Labels the phone field
Label 6	label8	Labels the password field
Label 7	label9	Labels the e-wallet field
Label 8	label10	Labels the search ID field
Label 9	label6	Labels the birthday field
Label 10	label7	Labels the address field
ComboBox 1	cboRole	Dropdown to select user role (Admin, Manager, Chef, Customer)
TextBox 1	txtName	Input field for user's name
TextBox 2	txtEmail	Input field for user's email address
TextBox 3	txtPhone	Input field for user's phone number
TextBox 4	txtSearchID	Input field for the target user's ID to search
TextBox 5	txtBirthday	Input field for birthday in YYYY-MM-DD format
TextBox 6	txtAddress	Input field for user's address
TextBox 7	txtPassword	Input field for user's password
TextBox 8	txtEWallet	Input field for e-wallet balance (enabled for Customer only)
Button 1	btnSearch	Searches the database for the specified ID and role

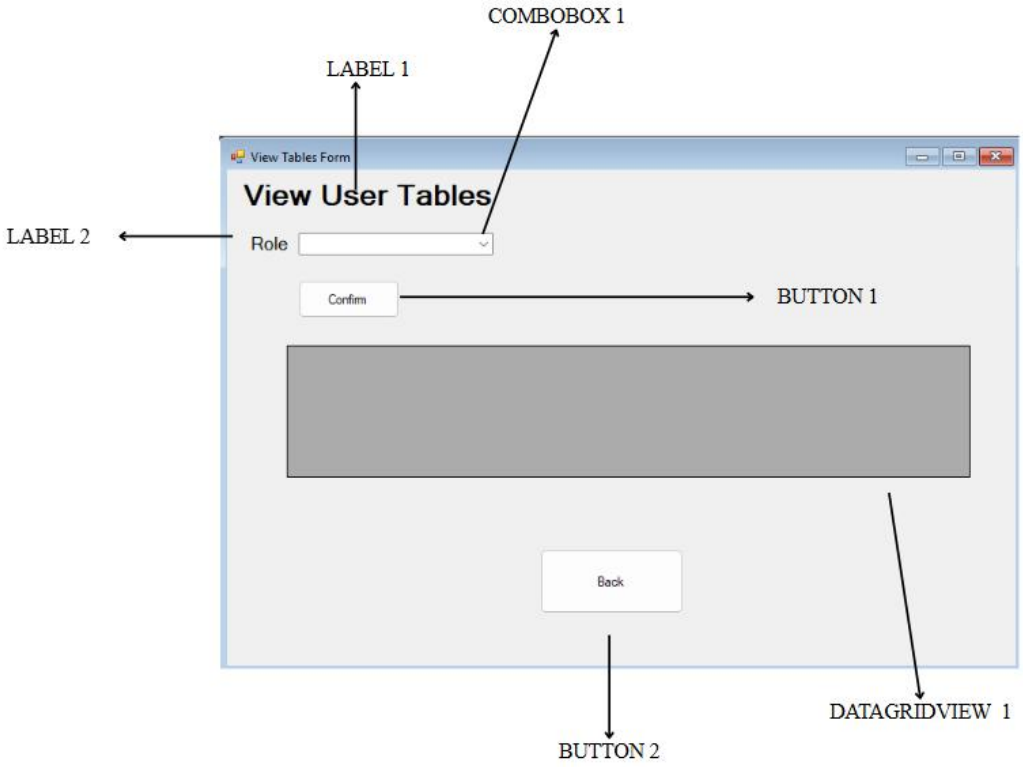
Button 2	btnDelete	Shows confirmation dialog then permanently deletes the user
Button 3	btnBack	Returns to the Admin Dashboard

1.2.5 View Sales Report Form



Control	Control Name	Description
Label 1	label1	Labels the month filter field
Label 2	label3	Labels the category filter dropdown
Label 3	lblTotalSales	Displays the total revenue calculated from filtered results
Label 4	label2	Labels the year field
Label 5	Label4	Labels the chef filter dropdown
TextBox 1	txtMonth	Input field for filtering by month name (e.g. January) or "All"
TextBox 2	txtYear	Input field for filtering by year (e.g. 2026) or "All"
ComboBox 1	cboCategory	Dropdown to filter by food category
ComboBox 2	cboChef	Dropdown to filter by chef name in the database
DataGridView 1	dgvSalesReport	Displays filtered sales report with order details and totals
Button 1	btnFilter	Applies selected filters and reloads the sales report
Button 2	btnBack	Returns to the Admin Dashboard
Button 3	btnClear	Resets all filters to default and reloads the report

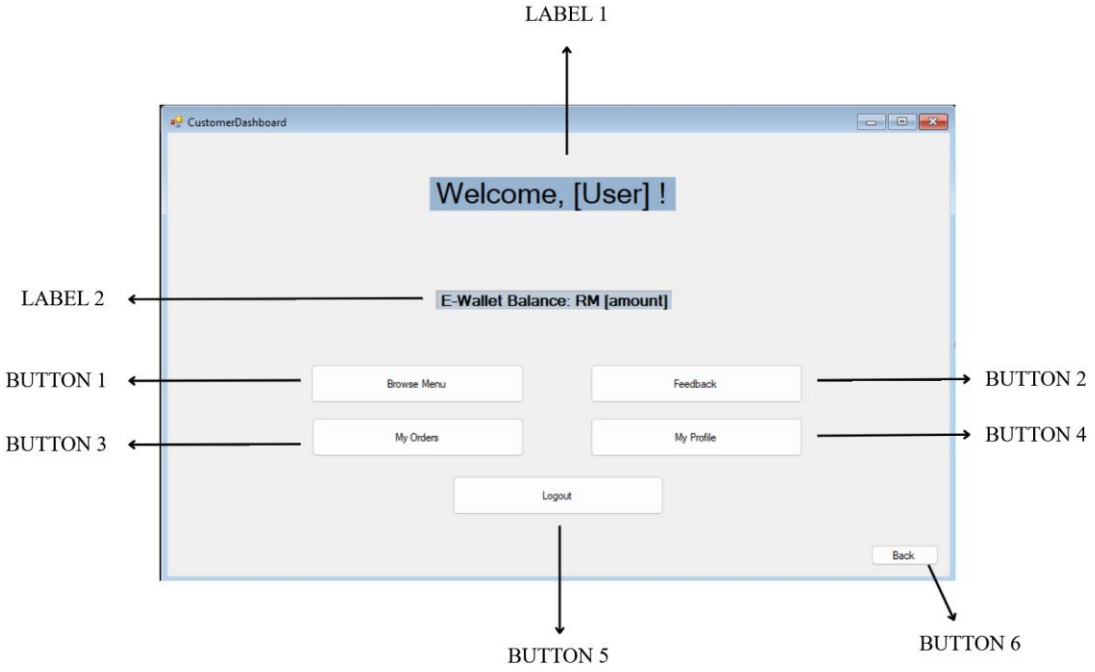
1.2.6 View Tables Form



Control	Control Name	Description
Label 1	label1	Displays the page title "View User Tables"
Label 2	label1	Labels the role selection dropdown
ComboBox 1	cboSelectTable	Dropdown to select which table to view (Admin, Manager, Chef, Customer)
DataGridView 1	dgvUserTables	Loads and displays data from the selected table
Button 1	btnConfirm	Applies selected filters and reloads the sales report
Button 2	btnBack	Returns to the Admin Dashboard

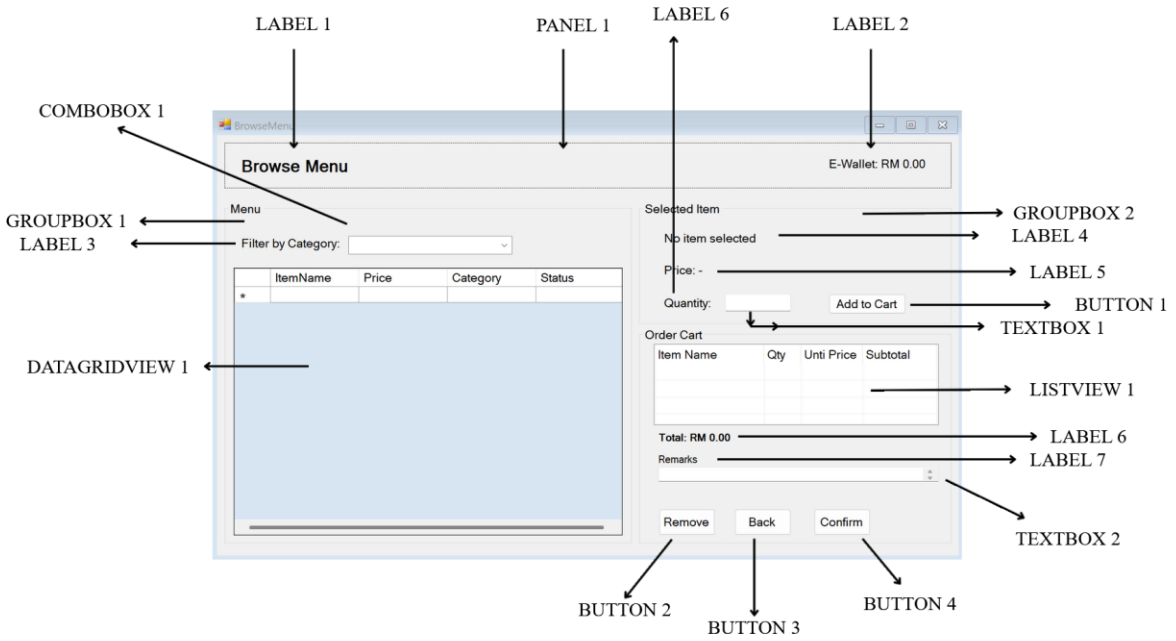
1.3 Customer Pages

1.3.1 Customer Dashboard



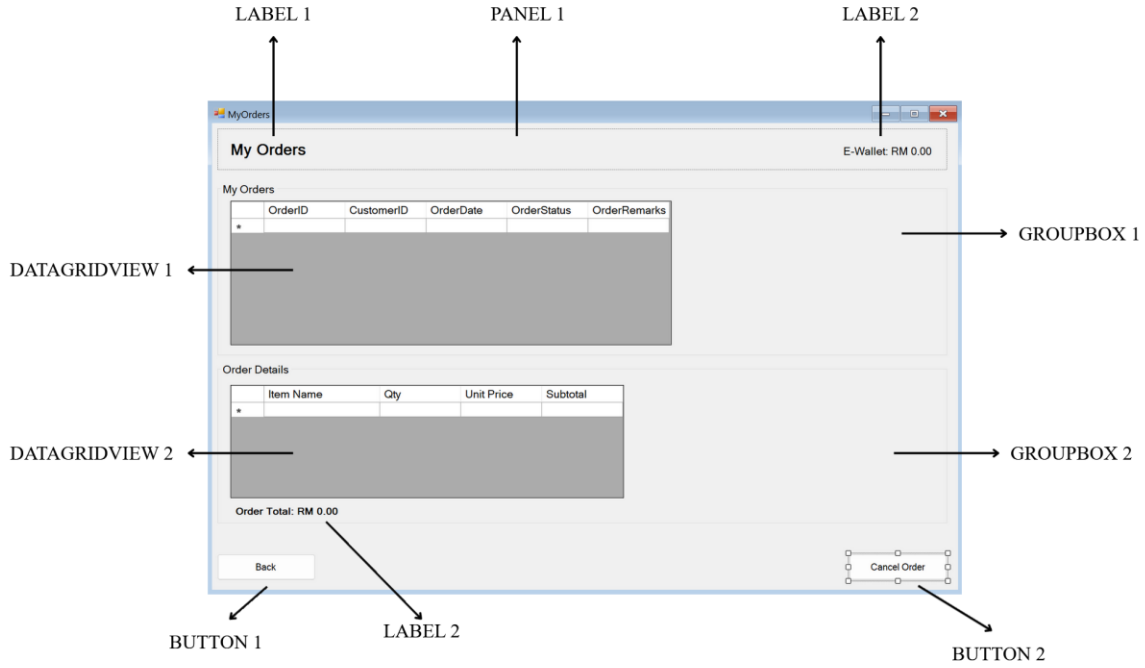
Control	Control Name	Description
Label 1	lblWelcome	Display Title
Label 2	lblWalletBalance	Display E-Wallet balance
Button 1	btnBrowseMenu	Navigate to the Browse Menu form
Button 2	btnFeedback	Navigate to the Feedback Form
Button 3	btnMyOrders	Navigate to the My Orders Form
Button 4	btnMyProfile	Navigate to the My Profile form
Button 5	btnLogout	Logout
Button 6	btnBack	Back

1.3.2 Browse Menu Page



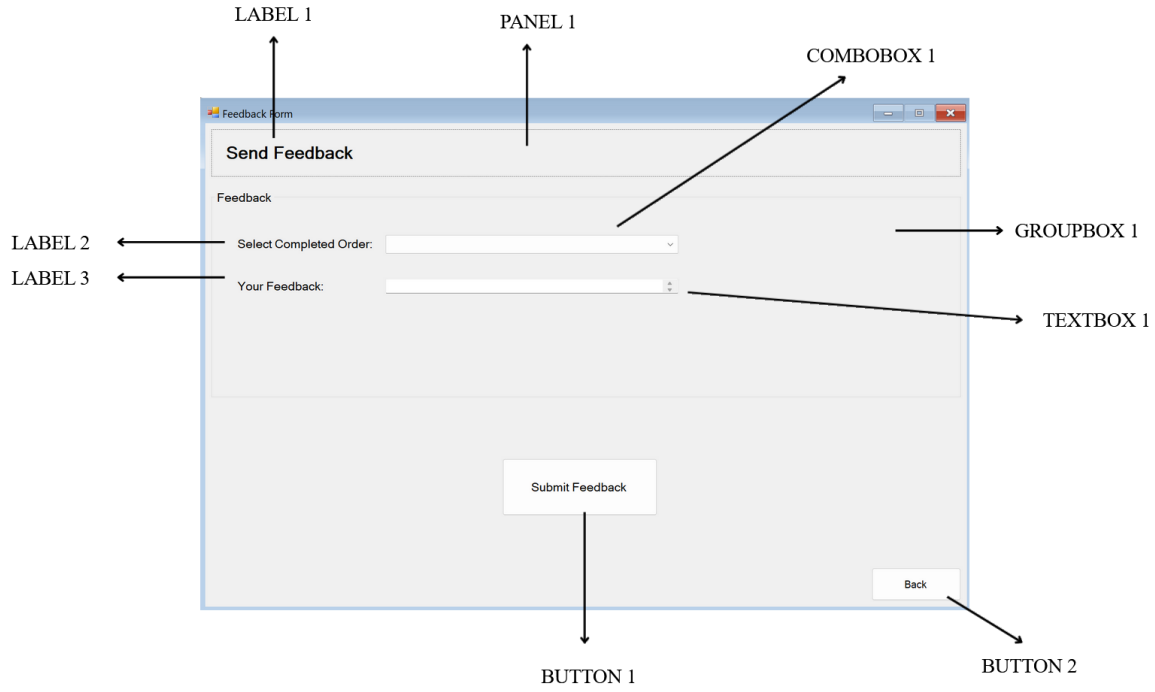
Control	Control Name	Description
Label 1	lblTItle	Display title page 'Browse Menu'
Label 2	lblWalletBalance	Display the user's E-wallet balance
Label 3	lblFilter	Label the related control
Label 4	lblMessage	Display the selected item from the menu
Label 5	lblPrice	Display the price of selected item
Label 6	lblTotal	Display the total price of all the selected items
Label 7	lblRemarks	Label related control
GroupBox 1	grpboxMenu	A section grouped for related controls, 'Menu'
GroupBox 2	grpboxSelectedItem	A section grouped for related controls, 'Selected Item'
GroupBox 3	grpboxOrderCart	A section grouped for related controls, 'Order Cart'
ComboBox 1	cboFilterbyCategory	Dropdown to filter menu by category (e.g: Breakfast, Lunch...)
TextBox 1	txtQty	Prompt user to input the quantity of selected item
TextBox 2	txtRemarks	Prompt user to input any kind of remarks regarding their order
DataGridView 1	dgvMenu	Shows all the menu available
ListView 1	lvOrderCart	Lists all the ordered item
Button 1	btnAddtoCart	Updates the order cart
Button 2	btnRemove	Removes a selected item from the order cart
Button 3	btnBack	Returns to previous page
Button 4	btnConfirm	Successfully add and complete the order

1.3.2 Customer's Order Page



Control	Control Name	Description
Label 1	lblTitle	Display the title page, 'My Orders'
Label 2	lblWalletBalance	Display the user's E-wallet banace
Label 3	lblTotal	Display the selected order total
GroupBox 1	grpboxMyOrders	A section grouped for related controls, 'My Orders'
GroupBox 2	grpboxOrderDetails	A section grouped for related controls, 'Order Details'
DataGridView 1	dgvMyOrders	Shows all the orders the user have made (In progress, Completed, Cancelled)
DataGridView 2	dgvOrderDetails	Shows the details of the selected order
Button 1	btnCancelOrder	Cancels the selected order from dgvMyOrders
Button 2	btnBack	Return to previous page

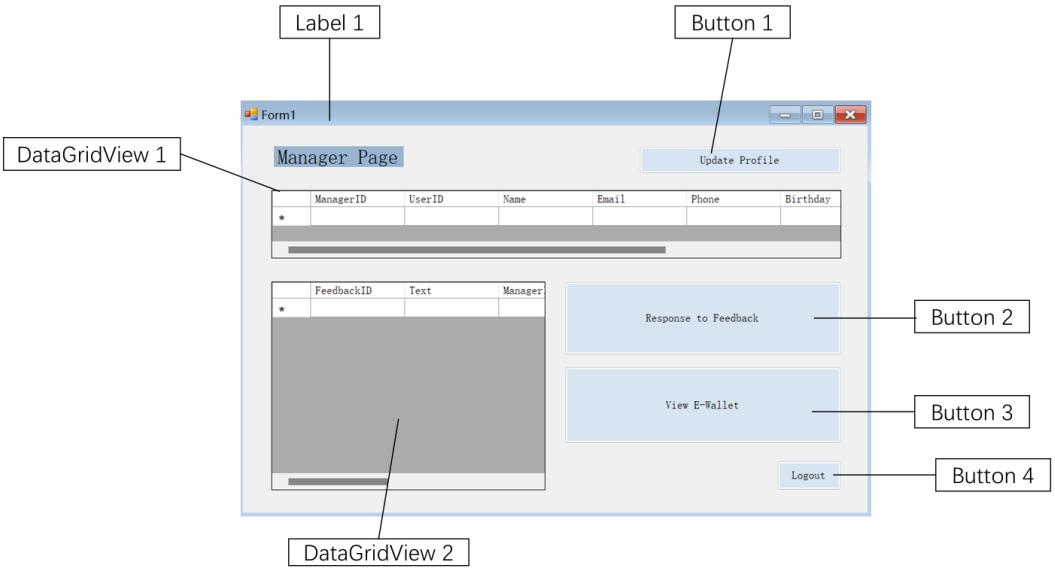
1.3.2 Customer's Feedback Page



Control	Control Name	Description
Label 1	lblTitle	Display the title page, 'Send Feedback'
Label 2	lblSelectedOrder	Label the related control, cboSelectedOrder
Label 3	lblFeedback	Label the related control, txtFeedback
GroupBox 1	grpboxFeedback	A section grouped for related controls, 'Feedback'
ComboBox 1	cboSelectedOrder	Dropdown of all completed orders the user have made
TextBox 1	txtFeedback	Prompt user to input their feedback regarding their order
Button 1	btnSubmitFeedback	Successfully submit the user's feedback
Button 2	btnBack	Return to previous page

1.4 Manager Pages

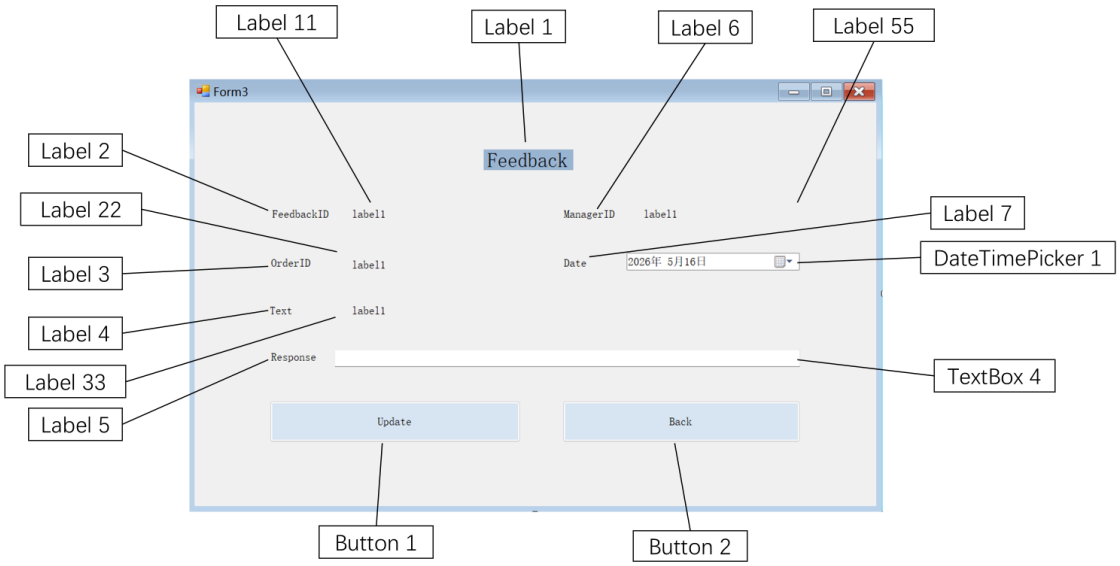
1.4.1 Home Page



Manager - Home Page

Control	Control Name	Description
Label 1	lbl1Title	Displays the page title “Manager Page”
Button 1	btnMProfile	Opens the Manager Profile information form
Button 2	btnFeedback	Opens the Feedback management form
Button 3	btnWallet	Opens the E-Wallet form
Button 4	btnLogout	Logs out from the system
DataGridView 1	dgvMProfile	Shows Manager’s basic information
DataGridView 2	dgvFeedback	Shows Feedback records

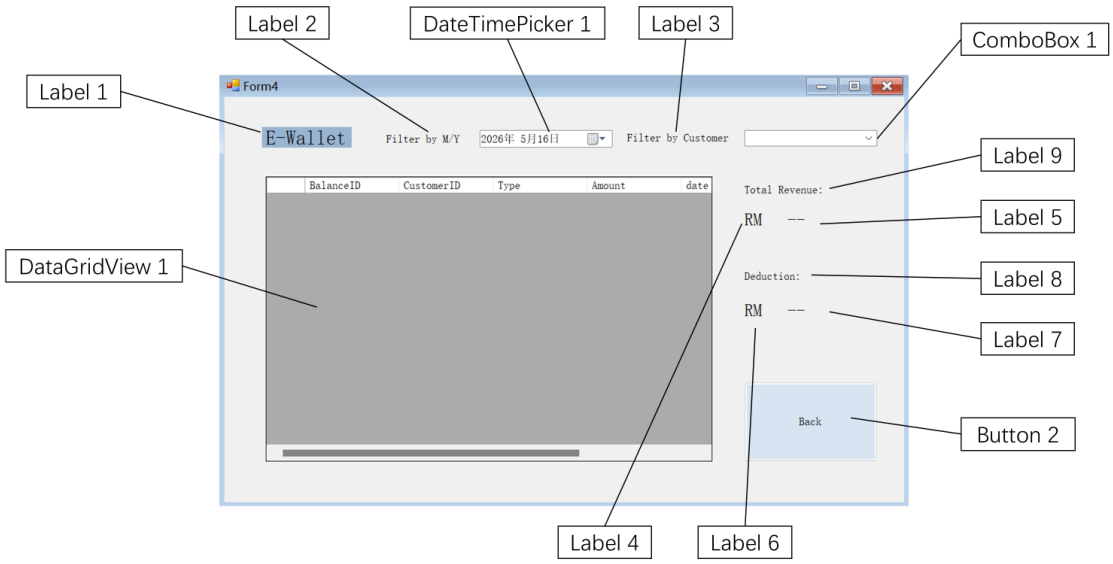
1.4.3 Manager Feedback Page



Manager – Feedback Response Page

Control	Control Name	Description
Label 1	lbl3Title	Displays the page title “Feedback”
Label 2	lblFID	To label related controls to the right
Label 3	lblFOID	
Label 4	lblFText	
Label 5	lblFResponse	
Label 6	lblFMID	
Label 7	lblFDate	
Label 11	labelFID	Controls of Feedback Information
Label 22	labelOID	
Label 33	labelText	
TextBox 4	tbFResponse	
Label 55	labelMID	
DateTimePicker 1	dtpFDate	
Button 1	btnM3Up	Updates feedback information
Button 2	btnM3Back	Returns to the previous page

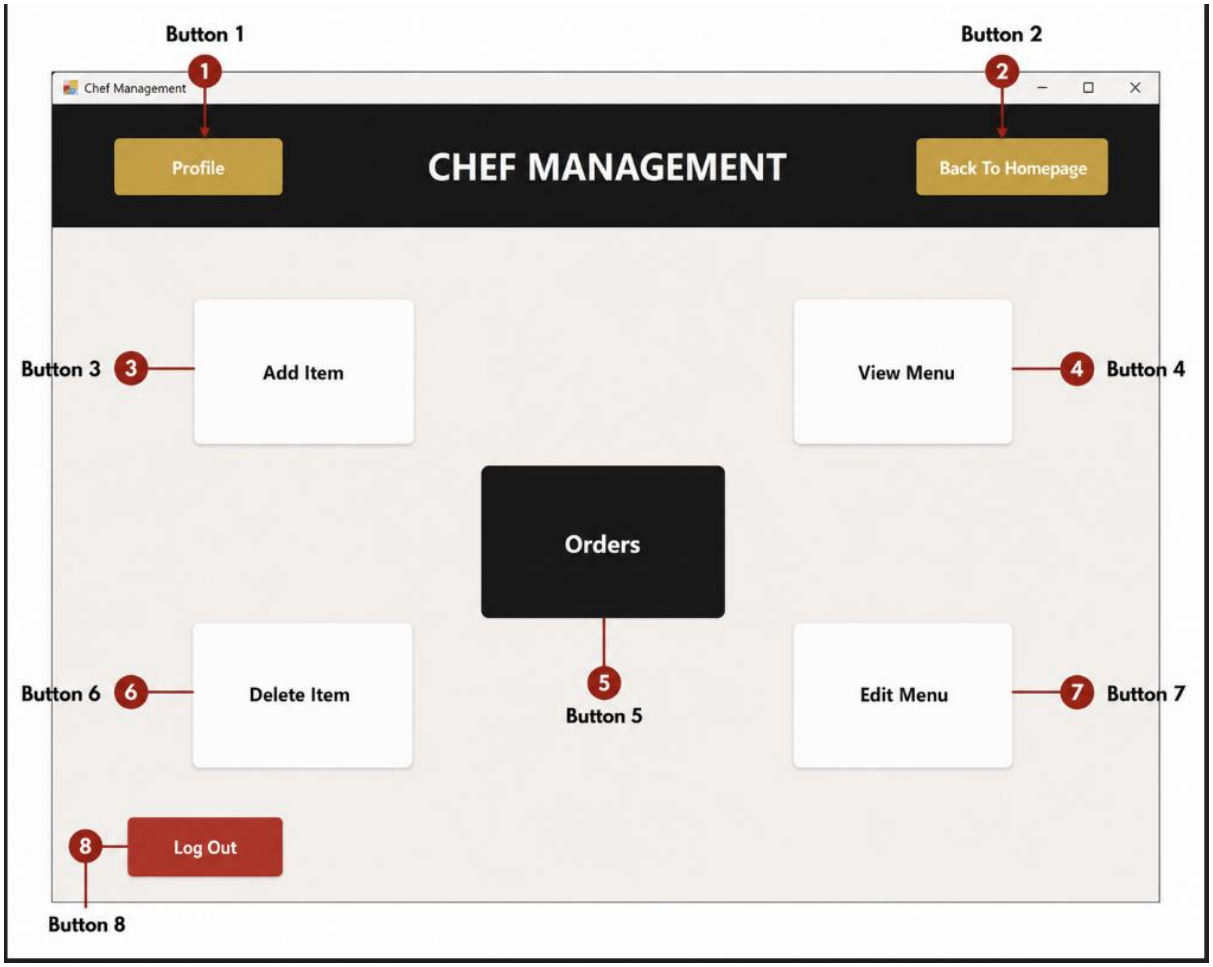
1.4.4 Manager e-wallet Page



Manager – E-Wallet Page

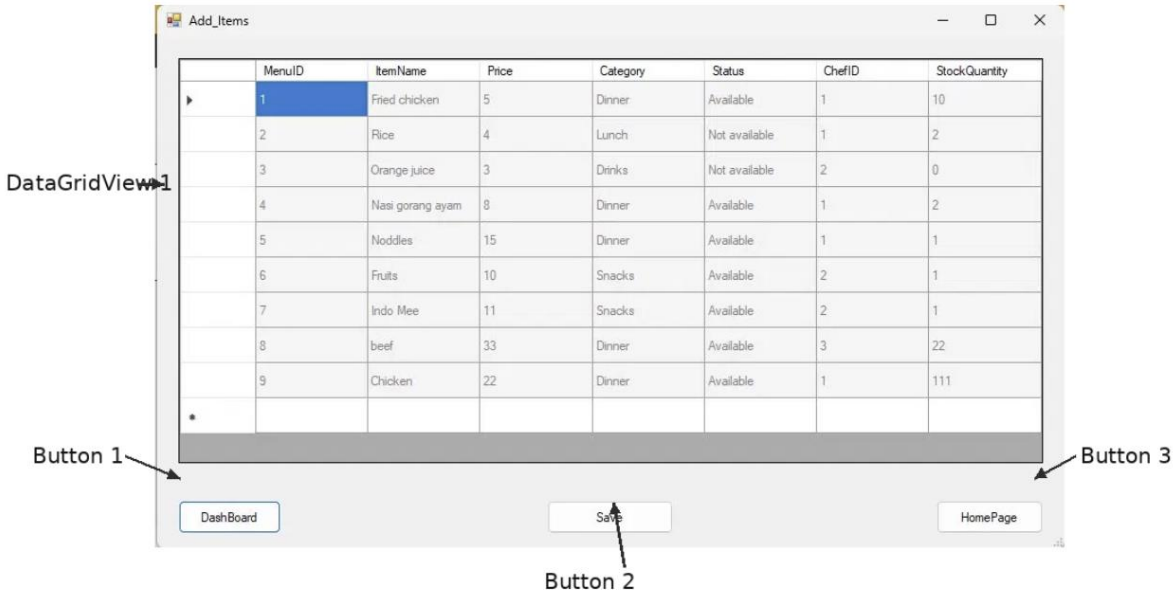
Control	Control Name	Description
DataGridView 1	dgvR	Displays E-Wallet balance records
Label 1	lbl4Title	Displays the page title “E-Wallet”
Label 2	lblRDate	Label for report generation date
Label 3	lblRType	Label for customer filter
Label 4	RM1	Display RM
Label 5	labelTR	Display total revenue
Label 6	RM1	Display RM
Label 7	labelID	Display deduction
Label 8	lblDeduction	Display text deduction
Label 9	lblTR	Display text total revenue
DataTimePicker 1	dtpRDate	Date selector for report generation
ComboBox 1	cbRType	Dropdown to filter by CustomerID
Button 1	btnM4Up	Saves the selected filter/report
Button 2	btnM4Back	Returns to the previous page

1.5 Chef Pages



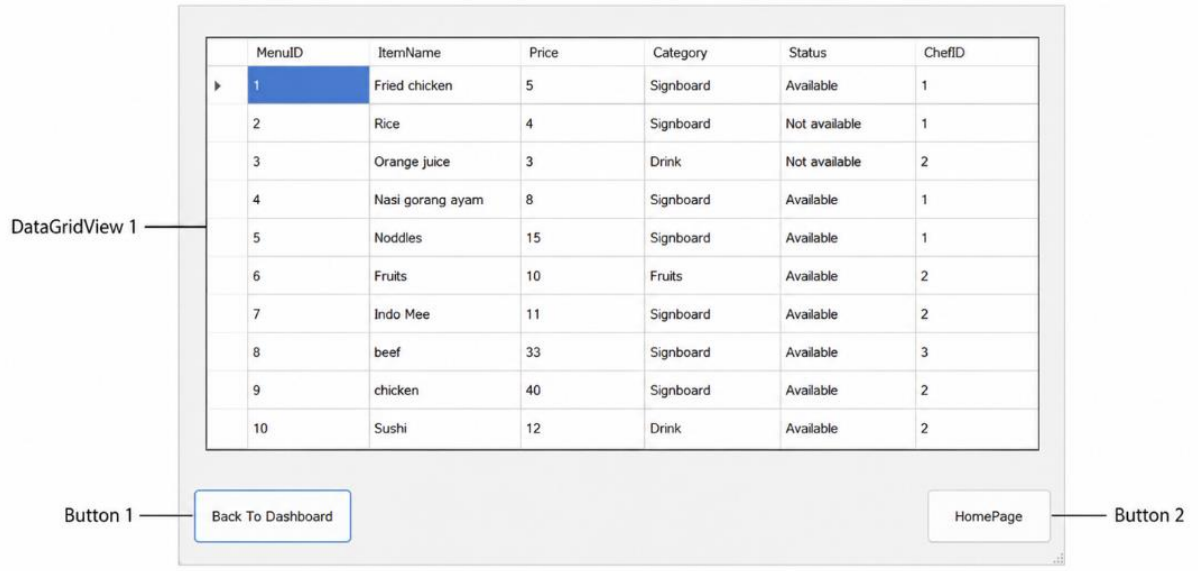
Chef – Dashboard Page

Control	Control Name	Description
Label 1	lblChefTitle	Displays the page title “CHEF MANAGEMENT”.
Button 1	btnProfile	Opens the user profile interface.
Button 2	btnHomePage	Returns the user to the application homepage.
Button 3	btnAdd	Adds a new item entry to the system.
Button 4	btnViewMenu	Opens a view to browse the full menu.
Button 5	btnOrders	Displays customer orders and their details.
Button 6	btnDelete	Removes a selected item from the menu.
Button 7	btnEditMenu	Opens the menu editor to modify items.
Button 8	btnLogOut	Logs the user out of the system.



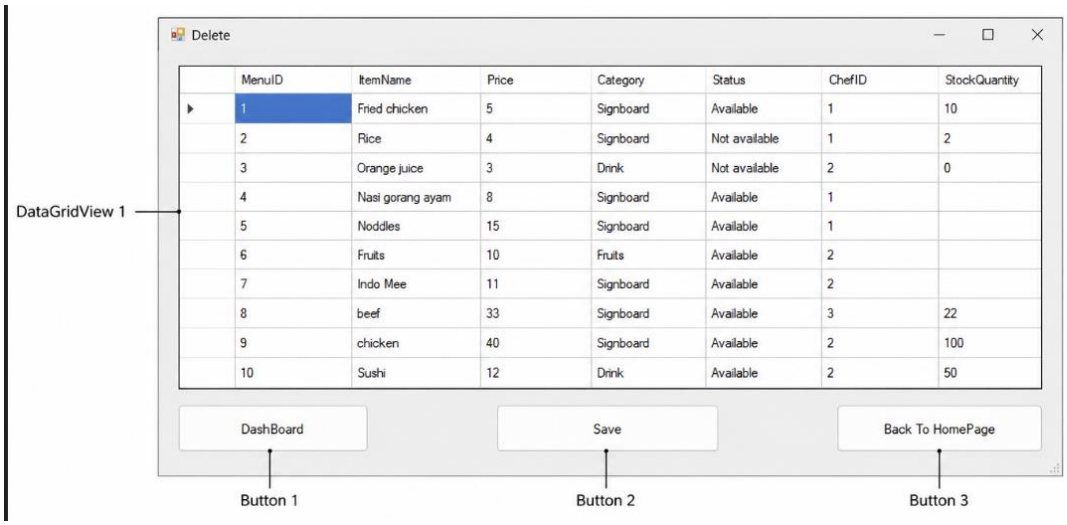
Chef – Add Items Page

Control	Control Name	Description
DataGridView 1	dgvStock	Displays all menu items with columns MenuID, ItemName, Price, Category, Status, ChefID, and StockQuantity. Existing rows are locked (read-only); a blank row at the bottom allows the chef to add a new item.
Button 1	btnDashboard	Returns the user back to the Chef Dashboard page.
Button 2	btnSave	Validates and saves any newly added rows to the Menu table in the database. Displays a success message with the count of rows inserted.
Button 3	btnHomePage	Closes the Add Items form and returns the user to the homepage.



Chef – Menu Page

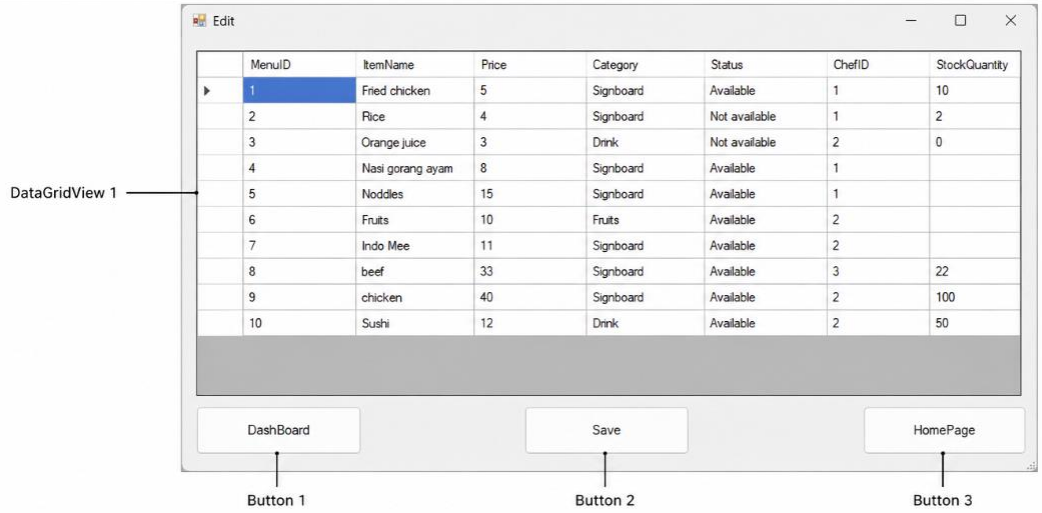
Control	Control Name	Description
DataGridView 1	dgvMenu	Displays the list of menu items, prices, categories, status, and Chef IDs.
Button 1	btnBackDashboard	Returns the user back to the Chef Dashboard page.
Button 2	btnHomePage	Returns the user to the application homepage.



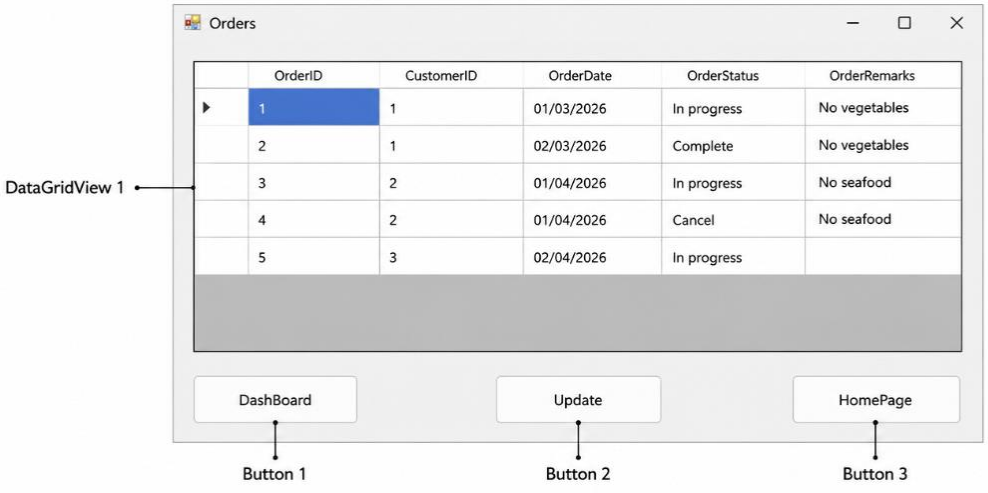
Chef – Delete Page

Control	Control Name	Description
DataGridView 1	dgvDeleteMenu	Displays the list of menu items and stock quantities for deletion management.
Button 1	btnDashboard	Returns the user back to the Dashboard page.
Button 2	btnSave	Saves the selected changes or deletion updates.
Button 3	btnHomePage	Returns the user to the application homepage.

Chef – Edit Page



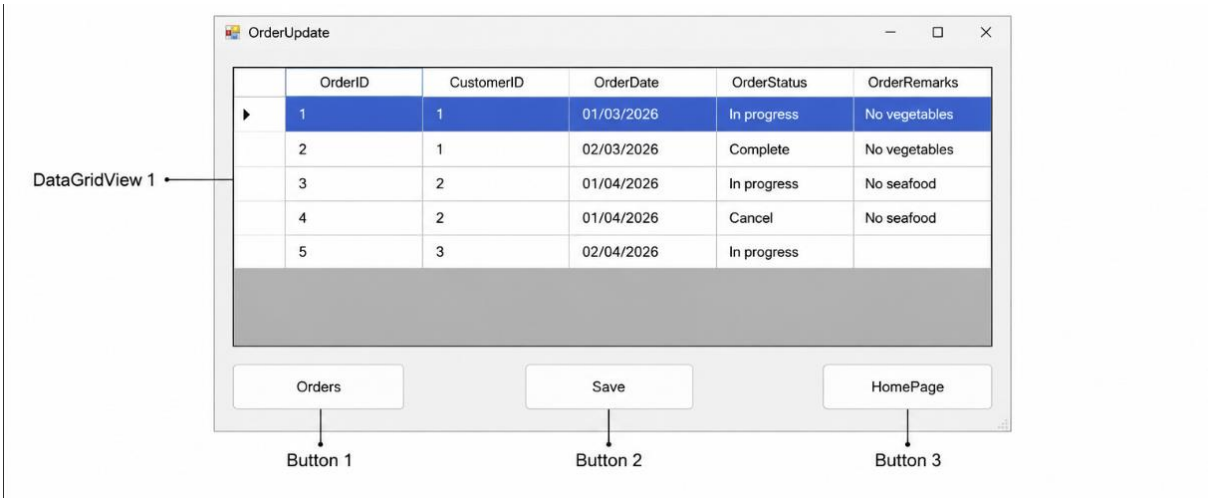
Control	Control Name	Description
DataGridView 1	dgvEditMenu	Displays the list of menu items and allows editing of item details and stock quantities.
Button 1	btnDashboard	Returns the user back to the Dashboard page.
Button 2	btnSave	Saves the edited menu item information.
Button 3	btnHomePage	Returns the user to the application homepage.



Chef – Orders Page

Control	Control Name	Description
DataGridView 1	dgvOrders	Displays the list of customer orders along with their details.

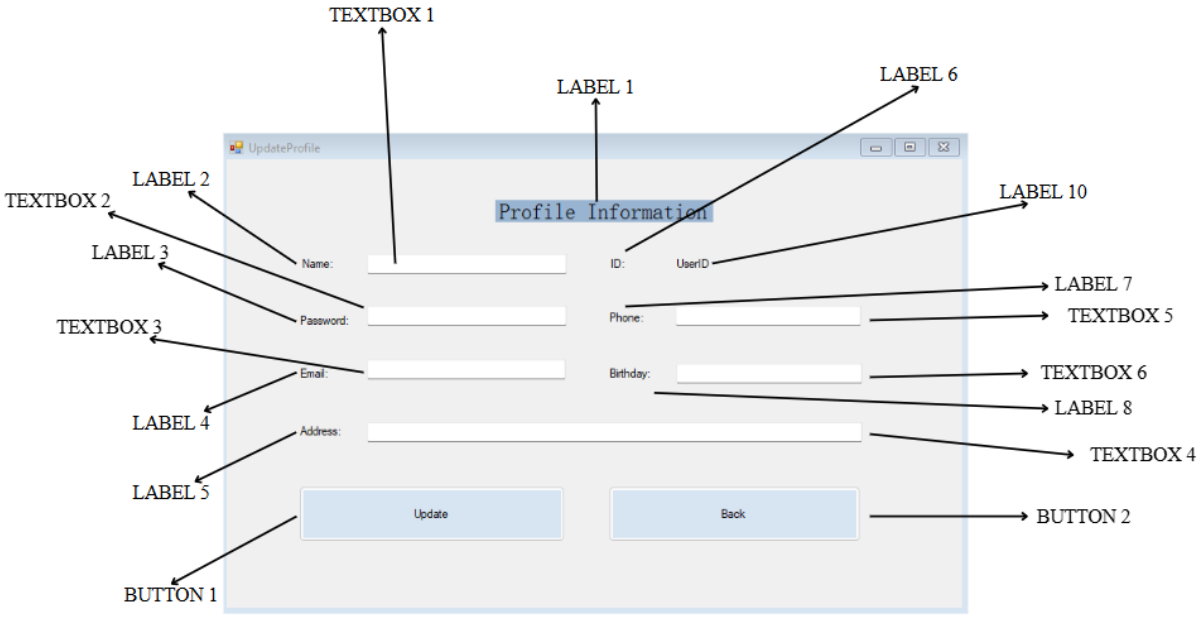
Button 1	btnDashboard	Returns the user back to the Dashboard page.
Button 2	btnUpdate	Updates the status of the selected order.
Button 3	btnHomePage	Returns the user to the application homepage.



Chef – Order Update Page

Control	Control Name	Description
DataGridView 1	dgvOrderUpdate	Displays the list of customer orders and allows order status updates.
Button 1	btnOrders	Returns the user back to the Orders page.
Button 2	btnSave	Saves the updated order information.
Button 3	btnHomePage	Returns the user to the application homepage.

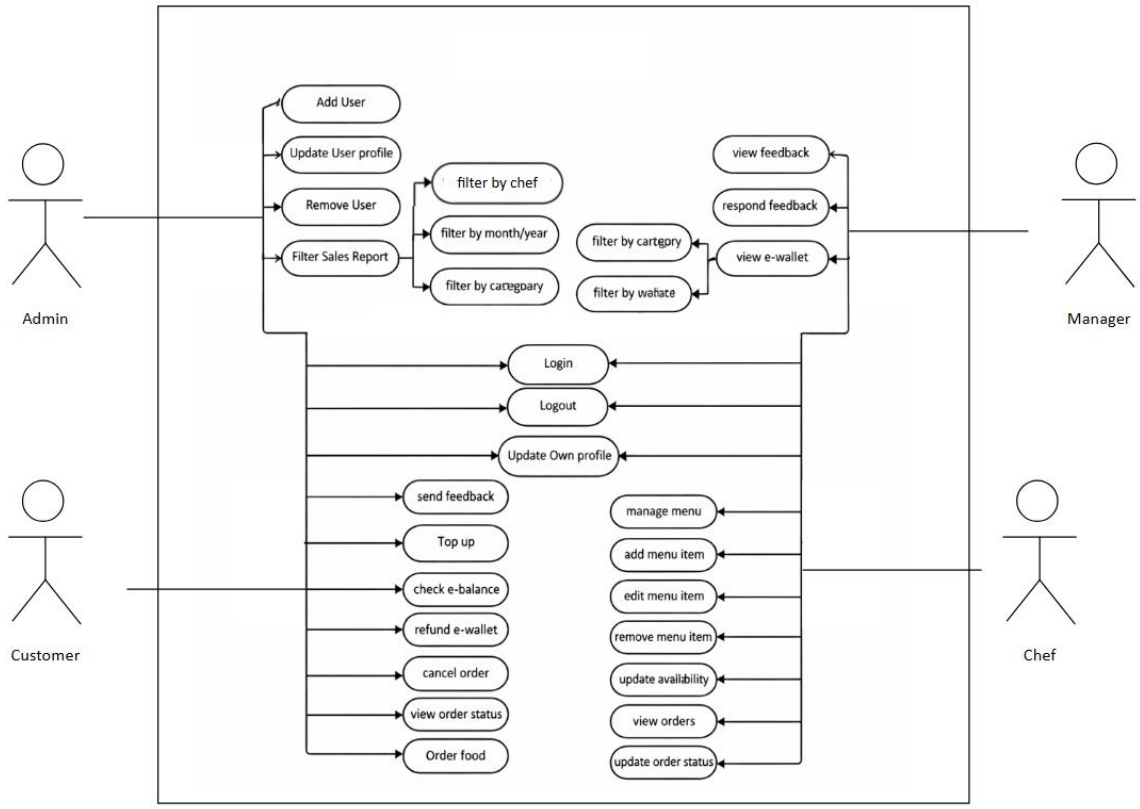
1.6 Update Profile Page



Manager – Profile Page

Control	Control Name	Description
Label 1	lbl2Title	Displays the page title “Profile Information”
Label 2	lblMName	To label related textboxes
Label 3	lblMPw	
Label 4	lblMEmail	
Label 5	lblMAdress	
Label 6	lblMID	
Label 7	lblMPhone	
Label 8	lblMBirth	
Label 9	lblUID	Shows the id of the current user
TextBox 1	tbMName	Loads current user data and is where the user inputs the new information to be updated
TextBox 2	tbMPw	
TextBox 3	tbMEmail	
TextBox 4	tbMAddress	
TextBox 5	tbMPhone	
TextBox6	tbMBirth	
Button 1	btnM2Up	Updates profile
Button 2	btnM2Back	Returns to the previous page

2. Use-Case Diagram



Use case diagram

3. Class Diagram

3.1 Admin

3.1.1 Admin Class Diagram

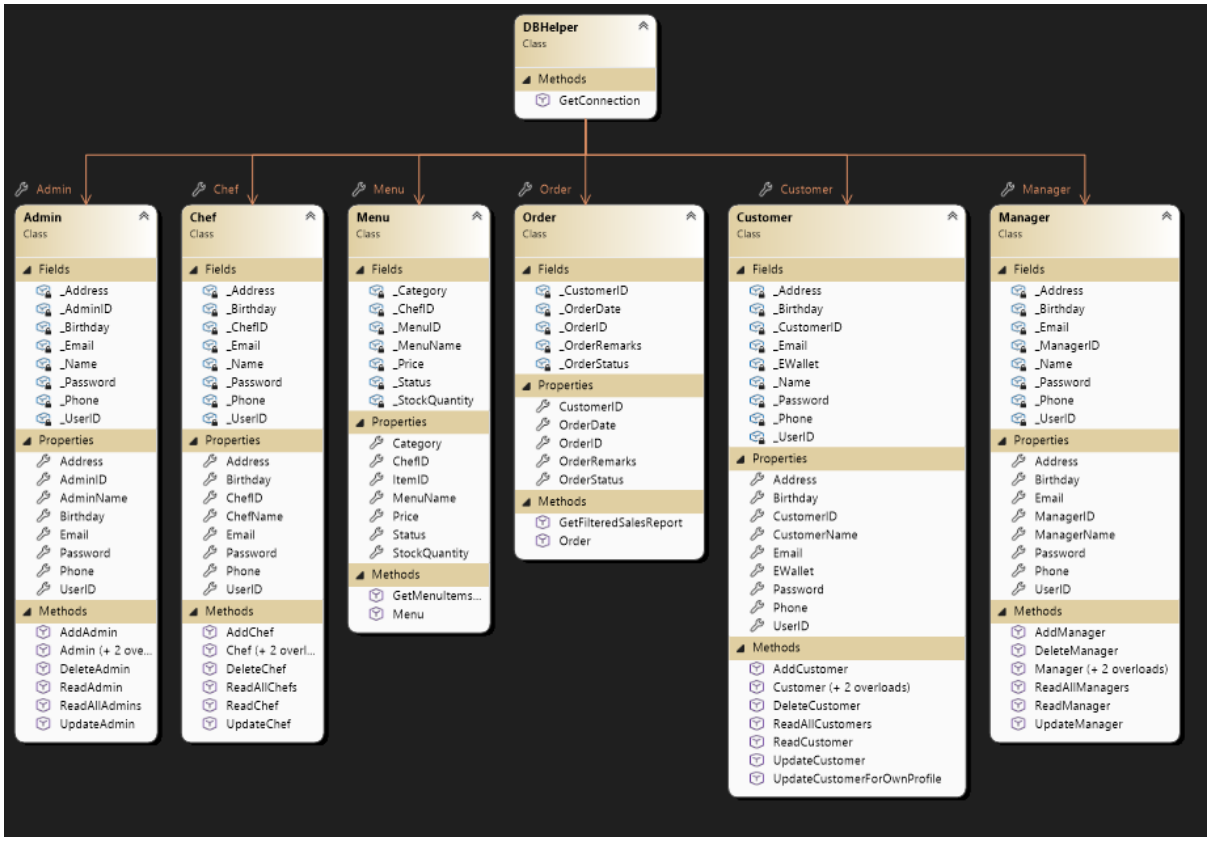


***Figure 3.2.0.1*:** Admin Class Diagram

3.1.2 DBHelper Class Methods

Class	**DBHelper**		
No.	Method Name	Description	Return Type
M1	GetConnection()	looks inside the App.config then it grabs the database connection string named "Gr8FoodDB". Lastly, it initializes and returns a brand-new SqlConnection object using that string.	DataTable

3.1.3 Admin Class Methods

Class	**Admin**		
No.	Method Name	Description	Return Type
M1	ReadAllAdmins()	Reads all admin data by using SELECT * FROM [dbo].[Admin];. Used for ViewTablesForm	DataTable

M2	ReadAdmin()	Reads a single record from admin table depending on the ID by using SELECT * FROM [dbo].[Admin] WHERE [AdminID] = @AdminID;. Used for RemoveUserForm, UpdateUserForm, and UpdateProfile	Admin
M3	AddAdmin()	Adds data into a new row to the admin table according to automated and user input by using INSERT INTO [dbo].[Admin] ([AdminID], [UserID], [Name], [Password], [Email], [Phone], [Birthday], [Address]) VALUES (@AdminID, @UserID, @Name, @Password, @Email, @Phone, @Birthday, @Address);. Used in AddUserForm	void
M4	UpdateAdmin()	Updates an existing record in the admin table according to the user's input by using UPDATE [dbo].[Admin] SET [UserID] = @UserID,[Name] = @Name,[Email] = @Email,[Phone] = @Phone,[Birthday] = @Birthday,[Address] = @Address,[Password] = @Password WHERE [AdminID] = @AdminID, Used in UpdateUserForm and UpdateProfile	void
M5	DeleteAdmin()	Removes a record from the admin table according to the ID inputted by the user by using DELETE FROM [dbo].[Admin] WHERE [AdminID] = @AdminID. Used for RemoveUserForm	void
M6	Admin(int id)	Initializes a minimal Admin object containing only the AdminID. Used when you only need an ID reference, like deleting a record, or searching a record using only the ID	N/A
M7	Admin(int userId, string name, string email, string phone, object birthday, string address, string password)	Initializes a new Admin object without AdminID. Used when creating/adding a brand-new admin account where the database hasn't assigned an ID yet.	N/A
M8	Admin(int adminId, int userId, string name, string email, string phone, object birthday, string	Creates a complete Admin object with all fields populated, including the AdminID. Used when updating an existing admin profile or loading one from the database.	N/A

	address, string password)		

3.1.4 Chef Class Methods

Class	**Chef**		
No.	Method Name	Description	Return Type
M1	ReadAllChefs()	Reads all Chef data by using SELECT * FROM [dbo].[Chef];. Used for ViewTablesForm	DataTable
M2	ReadChef()	Reads a single record from Chef table depending on the ID by using SELECT * FROM [dbo].[Chef] WHERE [ChefID] = @ChefID;. Used for RemoveUserForm, UpdateUserForm, and UpdateProfile	Chef
M3	AddChef()	Adds data into a new row to the Chef table according to automated and user input by using INSERT INTO [dbo].[Chef] ([ChefID], [UserID], [Name], [Password], [Email], [Phone], [Birthday], [Address]) VALUES (@ChefID, @UserID, @Name, @Password, @Email, @Phone, @Birthday, @Address);. Used in AddUserForm	void
M4	UpdateChef()	Updates an existing record in the chef table according to the user's input by using UPDATE [dbo].[Chef] SET [UserID] = @UserID,[Name] = @Name,[Email] = @Email,[Phone] = @Phone,[Birthday] = @Birthday,[Address] = @Address,[Password] = @Password WHERE [ChefID] = @ChefID, Used in UpdateUserForm and UpdateProfile	void
M5	DeleteChef()	Removes a record from the Chef table according to the ID inputted by the user by using DELETE FROM [dbo].[Chef] WHERE [ChefID] = @ChefID. Used for RemoveUserForm	void
M6	Chef(int id)	Initializes a Chef object storing only the ChefID. Used when you only need an ID reference, such as deleting a chef account, searching a chef account only using the ID.	N/A
M7	Chef(int userId, string name, string email, string	Initializes a new Chef object without ChefID. Used when creating/adding a	N/A

	phone, object birthday, string address, string password)	brand-new Chef account where the database hasn't assigned an ID yet.	
M8	Chef(int chefId, int userId, string name, string email, string phone, object birthday, string address, string password	Creates a complete Chef object with all fields populated, including the ChefID. Used when updating an existing Chef profile or loading one from the database.	N/A

3.1.5 Menu Class Methods

Class	**Menu**		
No.	Method Name	Description	Return Type
M1	GetMenuItemsWithChefDetails()	Reads data from the menu table joined with the chef table to get chef name by using SELECT m.[MenuID], m.[ItemName], m.[Price], c.[Name] AS [CreatedByChef] FROM [dbo].[Menu] m INNER JOIN [dbo].[Chef] c ON m.[ChefID] = c.[ChefID] ORDER BY m.[ItemName] ASC. Used in ViewSalesReportForm	DataTable
M2	Menu()	an empty, default constructor. It takes no parameters and doesn't initialize any fields when it runs. Used in ViewSalesReportForm	N/A

3.1.6 Order Class Methods

Class	**Menu**		
No.	Method Name	Description	Return Type
M1	GetFilteredSalesReport(object monthParam, object yearParam, string category, string chefName)	Reads data from the order table joined with the orderdetails table to get the	DataTable

		quantity to count the price, the menu table to get the item name and category, and the chef table to get the chef name by using SELECT o.[OrderID], o.[OrderDate], m.[ItemName], m.[Category], od.[Quantity], m.[Price] AS [UnitPrice], (od.[Quantity] * m.[Price]) AS [TotalPrice], c.[Name] AS [ChefName] FROM [dbo].[Order] o INNER JOIN [dbo].[OrderDetails] od ON o.[OrderID] = od.[OrderID] INNER JOIN [dbo].[Menu] m ON od.[MenuID] = m.[MenuID] LEFT JOIN [dbo].[Chef] c ON m.[ChefID] = c.[ChefID] WHERE o.[OrderStatus] = 'Complete', and depending on the parameter more query will be added, AND MONTH(o.[OrderDate]) = @Month, AND YEAR(o.[OrderDate]) = @Year, AND m.[Category] = @Category, and AND c.[Name] = @ChefName, and ending it with ORDER BY o.[OrderDate] DESC; used in ViewSalesReportForm	
M2	Order()	an empty, default constructor. It takes no parameters and doesn't initialize any fields when it runs. Used in ViewSalesReportForm	N/A

3.1.7 Customer Class Methods

Class	Chef		
No.	Method Name	Description	Return Type
M1	ReadAllCustomers()	Reads all Customer data by using SELECT * FROM [dbo].[Customer];. Used for ViewTablesForm	DataTable
M2	ReadCustomer()	Reads a single record from Customer table depending on the ID by using SELECT * FROM [dbo].[Customer] WHERE [CustomerID] = @CustomerID;. Used for RemoveUserForm, UpdateUserForm, and UpdateProfile	Customer
M3	AddCustomer()	Adds data into a new row to the Customer table according to automated and user input by using INSERT INTO [dbo].[Customer] ([CustomerID], [UserID], [Name], [Email], [Phone], [Birthday], [Address], [Password], [EWallet]) VALUES (@CustomerID, @UserID, @Name, @Email, @Phone, @Birthday, @Address, @Password, @EWallet);. Used in AddUserForm	void
M4	UpdateCustomer()	Updates an existing record in the Customer table according to the user's input by using UPDATE [dbo].[Customer] SET [UserID] = @UserID,[Name] = @Name,[Email] = @Email,[Phone] = @Phone,[Birthday] = @Birthday,[Address] = @Address,[Password] = @Password , [EWallet] = @EWallet WHERE [CustomerID] = @CustomerID, Used in UpdateUserForm	void
M5	DeleteCustomer()	Removes a record from the Customer table according to the ID inputted by the user by	void

		using DELETE FROM [dbo].[Customer] WHERE [CustomerID] = @CustomerID. Used for RemoveUserForm	
M6	UpdateCustomerForOwnProfile()	Updates an existing record in the customer table according to the user's input by using UPDATE [dbo].[Customer] SET [UserID] = @UserID, [Name] = @Name, [Email] = @Email, [Phone] = @Phone, [Birthday] = @Birthday, [Address] = @Address, [Password] = @Password WHERE [CustomerID] = @CustomerID, not able to update EWallet. Used in UpdateProfile	void
M7	Customer(int id)	Initializes a minimal customer object storing only the CustomerID. Used for operations that only require an ID, such as deleting an account.	
M8	Customer(int userId, string name, string email, string phone, object birthday, string address, string password)	Initializes a new Customer object without CustomerID. Used when creating a brand-new Customer account where the database hasn't assigned an ID yet.	
M9	Customer(int customerId, int userId, string name, string email, string phone, object birthday, string address, string password, int eWallet)	Creates a complete Customer object with all fields populated, including the CustomerID. Used when updating an existing Customer profile or loading one from the database.	

3.1.8 Manager Class Methods

Class	**Chef**		
No.	Method Name	Description	Return Type
M1	ReadAllManagers()	Reads all Managerdata by using SELECT * FROM [dbo].[Manager];. Used for ViewTablesForm	DataTable

M2	ReadManager()	Reads a single record from Manager table depending on the ID by using SELECT * FROM [dbo].[Manager] WHERE [ManagerID] = @ManagerID;. Used for RemoveUserForm, UpdateUserForm, and UpdateProfile	Manager
M3	AddManager()	Adds data into a new row to the Manager table according to automated and user input by using INSERT INTO [dbo].[Manager] ([ManagerID], [UserID], [Name], [Email], [Phone], [Birthday], [Address], [Password]) VALUES (@ManagerID, @UserID, @Name, @Email, @Phone, @Birthday, @Address, @Password);. Used in AddUserForm	void
M4	UpdateManager()	Updates an existing record in the Manager table according to the user's input by using UPDATE [dbo].[Manager] SET [UserID] = @UserID, [Name] = @Name, [Email] = @Email, [Phone] = @Phone, [Birthday] = @Birthday, [Address] = @Address, [Password] = @Password WHERE [ManagerID] = @ManagerID. Used in UpdateUserForm and UpdateProfile	void
M5	DeleteManager()	Removes a record from the Manager table according to the ID inputted by the user by using DELETE FROM [dbo].[Manager] WHERE [ManagerID] = @ManagerID. Used for RemoveUserForm	void
M6	Manager(int id)	Initializes a minimal Manager object storing only the ManagerID. Used for operations that only require an ID, such as deleting an account.	
M7	Manager(int userId, string name, string email, string phone, object birthday, string address, string password)	Initializes a new Manager object without ManagerID. Used when creating a brand-new Customer account where the database hasn't assigned an ID yet.	
M8	Manager(int managerId, int userId, string	Creates a complete Manager object with all fields populated, including the ManagerID. Used when updating an	

	name, string email, string phone, object birthday, string address, string password)	existing Customer profile or loading one from the database.	

3.2 Customer

3.2.1 Customer Class Diagram

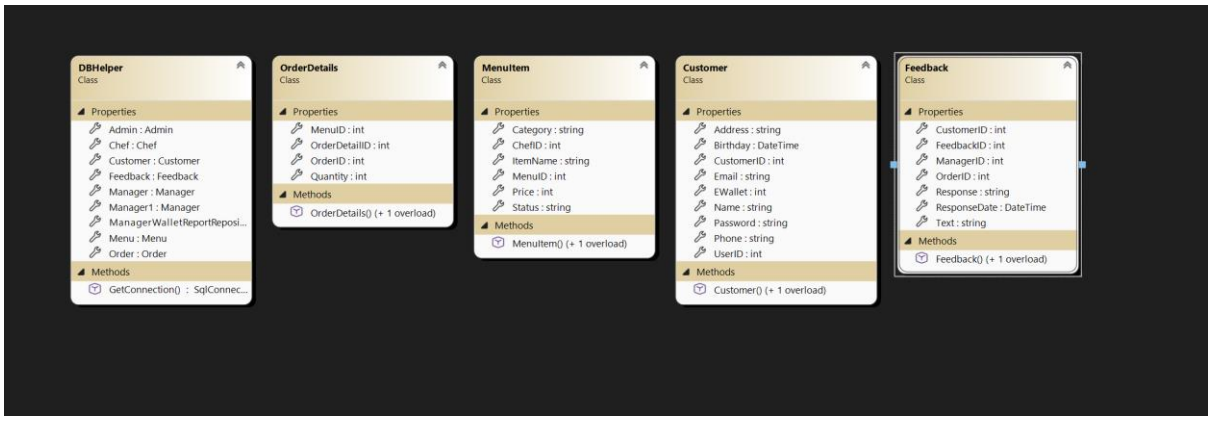


Figure: Customer UML Class Diagram

3.2.2 Customer Class

Class	Customer	
No.	Method Name	Return type
M1	Customer()	N/A
M2	Customer(int customerID, int userID, string name, string email, string phone, DateTime birthday, string address, string password, int eWallet)	N/A

M1 – The default empty constructor that initializes a blank Customer with no parameters. It takes no arguments and does not assign any values to the properties when called. This constructor is used when a Customer object needs to be created before any data is available to populate it.

M2 – The overloaded constructor that accepts all nine customer fields as parameters and assigns each one to its corresponding property inside the class. The customerID parameter maps to the CustomerID column, userID maps to UserID, name maps to Name, email maps to Email, phone maps to Phone, birthday maps to Birthday, address maps to Address, password maps to Password, and eWallet maps to EWallet. This constructor is used when loading customer data from the database using a SqlDataReader and populating a Customer object with the retrieved values.

3.2.3 MenuItem

Class	MenuItem	
No.	Method Name	Return type
M1	MenuItem()	N/A
M2	MenuItem(int menuID, string itemName, int price, string category, string status, int chefID)	N/A

- **M1 -** The default empty constructor that initializes a MenuItem object with no parameters. It takes no arguments and does not assign any values to the prperties wehn called. This constructor is used wehen a MenuItem object needs to be created before any menu data is available.
- **M2 –** The overloaded constructor that accepts all menu item fields as parameters and asigns each one to its corresponding property. The menuID parameter maps to MenuID column, itemName maps to ItemName, price maps to Price, category maps to Category, status maps to Status, and chefID maps to ChefID. This constructor is used when retrieving available menu items from the Menu table using the query **SELECT MenuID, ItemName, Category, Price FROM Menu WHERE Status = 'Available'** to display items in the Browse Menu form.

3.2.4 Order Details

Class	OrderDetails	
No.	Method Name	Return type
M1	OrderDetails()	N/A
M2	OrderDetails(int orderDetailID, int orderID, int menuID, int quantity)	N/A

- **M1 –** The default empty constructor that initializes a blank OrderDetails object with no parameters. It takes no arguments and does not assign any values to the properties when called. This constructor is used when an OrderDetails object needs to be created before any order data is available.
- **M2 –** The overloaded constructor that accepts all four order detail fields as parameters and assign each one to its corresponding preperty. The orderDetailID parameter maps to the OrderDetailID column, orderID maps to the OrderID, menuID maps to the MenuID, and quantity maps to Quantity. This constructor is used when inserting individual menu items into a customer's order using the query **INSERT INTO OrderDetails (OrderDetailID, OrderID, MenuID, Quantity) VALUES (@OrderDetailID, @OrderID, @MenuID, @Quantity)** during the order confirmation process in the Browse Menu form.

3.2.5 Customer Feedback

Class	Feedback	
No.	Method Name	Return type

M1	Feedback()	N/A
M2	Feedback(int feedbackID, string text, int managerID, string response, DateTime responseDate, int orderID, int customerID)	N/A

- **M1 –** The default empty constructor that initializes a blank Feedback object with no parameters. It takes no arguments and does not assign any values to the properties when called. This constructor is used wehn a Feedback object needs to be created before any feedback data is available.
- **M2 –** The overloaded constructor that accepts all seven feedback fields as parameters and assigns each one to its corresponding preperty. The feedbackID parameter maps to the FeedbackID column, text maps to Text, managerID maps to ManagerID, response maps to Response, responseDate maps to ResponseDate, orderID maps to OrderID, and customerID maps to CustomerID. This constructor is used when inserting a new feedback record into the Feedback table using the query **INSERT INTO Feedback (FeedbackID, Text, OrderID, CustomerID) VALUES (@FeedbackID, @Text, @OrderID, @CustomerID)** in the Feedback form after the customer submits their feedback for a completed order.

3.2.6 DBHelper

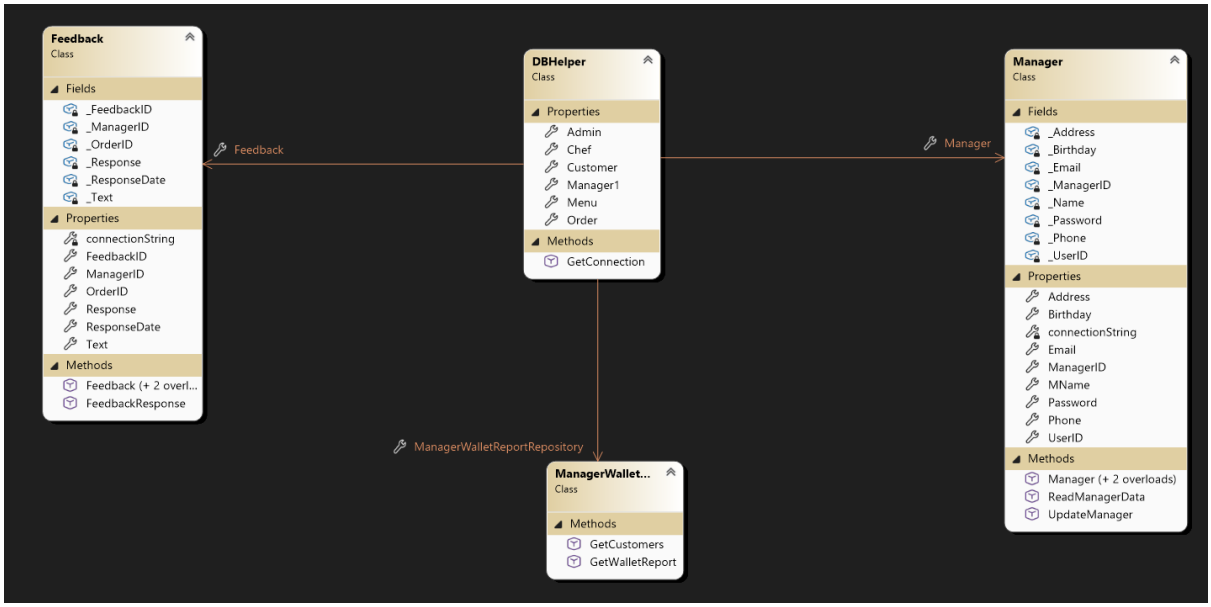
Class	**DBHelper**	
No.	Method Name	Return type
M1	DBHelper()	N/A
M2	GetConnection()	N/A

M1 – The default constructor that initializes a new DBHelper class. It takes no parameters and sets up the object so that GetConnection() can be called on it. Since GetConnection() is an instance method, a DBHelper object must always be created first before calling it. For example. DBHelper db = new DBHelper();

- **M2 –** An instance method that read the connection string named Gr8FoodDB from the App.config configuration file using ConfigurationManager.ConnectionStrings["Gr8FoodDB"].ConnectionString and returns a new SqlConnection object initialized with that connection string. The returned connection must then be opened by calling conn.Open() before any SQL commands can be executed.
- The M2 method is called in every form across the Customer module whenever a database operation is required. For example, DBHelper db = new DBHelper(); SqlConnection conn = db.GetConnection(); conn.Open();. This ensures th if the database server details change, only the App.config file needs to be updated rather than every individual form.

3.3 Manager

3.3.1 Class Diagram



Manager Class Diagram

3.3.2 Feedback class

Class	Feedback	
No.	Method Name	Return type
1	Feedback(int id)	N/A
2	Feedback(int id, string text, int? managerId, string response, DateTime? responseDate, int? orderId)	N/A
3	Feedback(int id, string text, int? managerId, string response, DateTime? responseDate, int? orderId, bool forUpdate)	N/A
4	FeedbackResponse()	void

In the Manager class that corresponds directly to the database table for Feedback.

Constructor

The constructor **M1** is used when you only need to work with a specific feedback record by its identifier, such as preparing to delete or update that entry. The overloaded constructor **M2** allows you to create a new feedback object with all its fields populated, which is useful for inserting new records into the database. Extending this, **M3** is tailored for update operations, where the extra forUpdate flag distinguishes the constructor's purpose and ensures the object is prepared for modification rather than insertion.

Core function: update profile

Finally, **M4** is the method that executes the actual update logic: it opens a database connection, builds an SQL UPDATE statement with parameters, and commits the changes so that the feedback record in the database reflects the new text, manager assignment, response, date, and order linkage.

3.3.3 Manager class

Class	Manager	
No.	Method Name	Return type
1	Manager(int id)	N/A
2	Manager(int userId, string name, string email, string phone, DateTime? birthday, string address, string password)	N/A
3	Manager(int id, int userId, string name, string email, string phone, DateTime? birthday, string address, string password)	N/A
4	ReadManagerData()	DataTable

Constructor

The constructor **M1** is used when you only need to reference a manager by their unique identifier, such as retrieving or deleting a record. The overloaded constructor **M2** allows you to create a new manager object with all the necessary details filled in, which is typically used when inserting a new record into the database. Extending this, **M3** is intended for updating operations, where both the manager's ID and user ID are specified alongside the other fields, ensuring the correct record is modified.

Core function: ReadManagerData()

Finally, **M4** is the method that executes the actual database read logic: it reads a record, by executing an SQL statement and shows it in the manager home screen.

3.3.4 ManagerWalletReportRepository class

In the ManagerWalletReportRepository class, the methods are designed specifically to support filtering e-wallet reports by customer and date.

Class	ManagerWalletReportRepository	
No.	Method Name	Return type
1	GetCustomers()	DataTable
2	GetWalletReport(int customerId, int year, int month)	DataTable

Core function 1: filter by customer

The constructor **M1** returns a DataTable containing all customer records, which is typically used to populate the combo box in Form 4 so managers can select a specific customer for filtering.

Core function 2: filter by month/year

The second method **M2** also returns a DataTable, but this one is filled with transaction records filtered by the given customer ID and the selected year and month. This allows the e-wallet report grid to display only the relevant entries, making it possible to calculate totals, deductions, and revenue for that specific customer and time period.

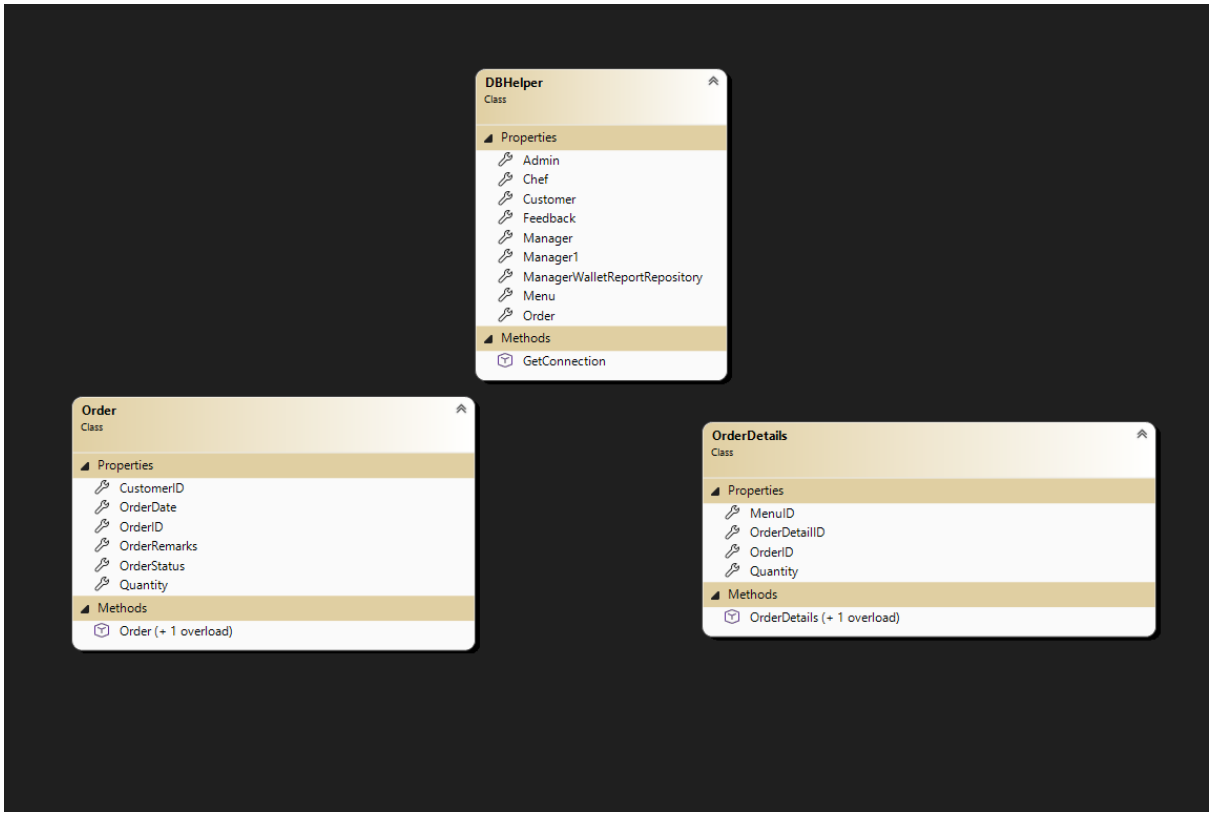
3.3.5 DBHelper

Class	DBHelper	
No.	Method Name	Return type
M1	DBHelper()	N/A
M2	GetConnection()	N/A

- **M1** initializes a new DBHelper class(no parameters)
- **M2 –** An instance method that read the connection string named Gr8FoodDB from the App.config configuration file using ConfigurationManager.ConnectionStrings["Gr8FoodDB"].ConnectionString and returns a new SqlConnection object initialized with that connection string. The returned connection must then be opened by calling conn.Open() before any SQL commands can be executed.
- Method and function are same for each role

3.4 Chef

3.4.1 Class Diagram



Chef Class Diagram

3.4.2 DBHelper class

Class	Method Name	Return type
DBHelper	GetConnection()	SqlConnection

Explanation

The DBHelper class is responsible for database connection support. Its method GetConnection() creates and returns a SqlConnection object, which is used to connect the program to the SQL Server database. In .NET, SqlConnection represents a connection/session with a SQL Server database.

3.4.3 Order class

Class	Method Name	Return type
Order	Order()	N/A
Order	Order(int orderID, int customerID, DateTime orderDate, string orderStatus, string orderRemarks, int quantity)	N/A

Explanation

The Order class represents the main order information. The first constructor, Order (), creates an empty order object. The second constructor is an overloaded constructor that creates an order object with full order details, including order ID, customer ID, order date, order status, order remarks, and quantity.

Constructors do not have a return type because they are used to initialize a new object when it is created. Microsoft's C# documentation explains that a constructor has the same name as its class and does not include a return type.

3.4.4 Order Details class

Class	Method Name	Return type
OrderDetails	OrderDetails()	N/A
OrderDetails	OrderDetails(int orderDetailID, int orderID, int menuID, int quantity)	N/A

Explanation

The OrderDetails class stores the detailed information for items inside an order. The empty constructor OrderDetails() creates a blank order-detail object. The overloaded constructor creates an order-detail object with the order detail ID, order ID, menu ID, and quantity.

This class is connected to the Order class because one order can have related order details. For example, the Order class stores the general order record, while OrderDetails stores the specific menu items and quantities linked to that order.

4. Test Plan and Test Cases

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
General (All Roles)					
1	Log In	Verify that a user can log in using a valid Role ID and password.	The user is authenticated and redirected to the dashboard that corresponds to their role (Admin, Manager, Chef, or Customer).	The application successfully authenticated the user and displayed the correct role-specific dashboard.	None
2	Log Out	Ensure that a user can log out and be returned to the login page.	The user is logged out, and the login form is shown. The session is ended and no further access is possible without re-authenticating.	The user was redirected to the login form, and the session was closed successfully.	None
Admin					
3	Register New User	Check that an admin can register a new user by selecting a role and filling in the required details.	The admin successfully creates a new account for the selected role (Admin, Manager, Chef, or Customer). The new record is saved in the corresponding database table, and the user can log in.	The new user record was inserted into the database. A success message was displayed, and the form fields were cleared.	None

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
4	Update User Profile (Admin)	Verify that an admin can search for an existing user by role and ID, modify their details, and save the changes.	The admin searches by role and ID, loads the existing data into the form, edits the fields, and the updated record is saved to the database.	The user record was updated in the database. A success message confirmed the change.	None
5	Remove User	Verify that an admin can search for a user by role and ID, view their details, and permanently delete them from the system.	The admin locates the user, reviews the displayed profile details, confirms deletion, and the record is removed from the database.	The selected user record was permanently deleted from the database after confirmation. A success message was shown.	None
6	View User Tables	Ensure that an admin can view the full data table for each role (Admin, Manager, Chef, Customer) using a dropdown selector.	After selecting a role from the dropdown and clicking Confirm, the corresponding database table is displayed in the data grid with all columns.	The correct table data was retrieved from the database and displayed in the data grid for each selected role.	None
7	View & Filter Sales Report	Check that an admin can view a sales report of completed orders and filter by month, year,	The report grid updates to show only completed orders matching the selected filters. The total revenue label calculates and	The filtered sales report loaded correctly with accurate order data and the total revenue was summed	None

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
		category, and chef name.	displays the correct summed amount.	and displayed correctly.	
8	Phone Number Validation – Add/Update User	Verify that the system enforces role-specific phone number rules: exactly 11 digits, digits only, and the correct starting digit per role.	An error message is displayed if the phone number is not 11 digits, contains non-numeric characters, or does not start with the correct digit for the selected role.	Validation errors were shown for invalid phone inputs. Valid phone numbers with the correct prefix and length were accepted.	None
Customer					
9	View E-Wallet Balance	Verify that the customer's current e-wallet balance is loaded and displayed correctly on the dashboard and Browse Menu screen.	The e-wallet balance label shows the correct value fetched from the Customer table in the database.	The correct e-wallet balance was retrieved from the database and displayed on both the dashboard and the Browse Menu form.	None
10	Browse Menu by Category	Ensure that a customer can filter the available menu items by category using the dropdown selector.	Only menu items with Status = 'Available' belonging to the selected category are displayed in the data grid. Selecting 'All' shows every available item.	The menu grid filtered correctly by category. Items with Status 'Available' were displayed and unavailable items were excluded.	None
11	Add Item to Cart &	Check that a customer can	A valid quantity (1–99) adds the item to	Valid items and quantities	None

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
	Validate Quantity	select a menu item, enter a quantity, and add it to the cart. Verify that quantity validation prevents invalid entries.	the cart and updates the total. An error message is shown if no item is selected, the field is empty, the quantity is non-numeric, zero or negative, or exceeds 99.	were added to the cart and the total updated correctly. All invalid quantity inputs displayed appropriate error messages.	
12	Remove Item from Cart	Verify that a customer can remove a selected item from the cart and that the order total is recalculated.	After confirming removal, the selected item is removed from the cart list, and the total amount is reduced by the item's subtotal.	The selected cart item was removed after confirmation, and the total amount was recalculated and displayed correctly.	None
13	Place Order (Insufficient Balance)	Verify that the system prevents an order from being placed when the cart total exceeds the customer's e-wallet balance.	A warning message is shown stating insufficient balance, and the order is not saved to the database.	The system correctly detected the insufficient balance and displayed a warning. The order was not created in the database.	None
14	Place Order (Sufficient Balance)	Verify that a customer can confirm and place an order when the e-wallet balance is sufficient.	The order is inserted into the Order and OrderDetails tables. The e-wallet balance is deducted by the total amount, and a Balance record is created. A	The order was saved to the database, the e-wallet was deducted, the balance log was recorded, and a success	None

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
			success message is shown.	message was displayed.	
15	Cancel Order & Refund	Ensure that a customer can cancel an 'In progress' order and receive a refund to their e-wallet.	The order status is updated to 'Cancel', the order total is refunded to the e-wallet, and a refund record is inserted into the Balance table.	The order status was changed to 'Cancel', the wallet balance was restored by the correct amount, and the refund was recorded in the Balance table.	None
16	View Order History & Order Details	Verify that a customer can view all their past orders with status colour-coding and click an order to see its item breakdown.	All orders belonging to the customer are loaded sorted by date. Rows are colour-coded by status. Clicking an order displays its items, quantities, and subtotals in the details grid.	The order list loaded with correct data and colour coding. Selecting an order displayed the correct item details and calculated total.	None
17	Submit Feedback – Validation	Check that the feedback form validates order must be selected, text must be 10–100 characters, and duplicate feedback for the same order is rejected.	Error messages are shown for missing order selection, text too short (< 10 chars), text too long (> 100 chars), and if feedback for that order already exists.	All validation rules were enforced correctly. Only feedback meeting all conditions was accepted and saved to the Feedback table.	None
18	Update Customer	Verify that the profile update	Appropriate error messages are shown	All validation checks were	None

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
	Profile – Email & Phone Validation	form validates email format (must contain '@' and '.'), phone length (10–20 digits, numeric only), and birthday (not in future, not before 1900).	for an invalid email, non-numeric or out-of-range phone, future birthday, or password shorter than 6 characters. Valid data is saved to the database.	triggered correctly for invalid inputs. Valid profile data was updated in the Customer table successfully.	
Chef					
19	View Menu	Verify that a chef can view all menu items from the Menu table in a read-only data grid.	All columns (MenuID, ItemName, Price, Category, Status, ChefID, StockQuantity) are displayed. The grid is read-only and no add/delete operations are permitted.	All menu items were loaded from the database and displayed correctly in the read-only grid.	None
20	Add Menu Item	Check that a chef can add a new menu item by filling in the new row at the bottom of the grid and saving.	The new row data is validated (ItemName not empty, Price positive number, ChefID is integer), inserted into the Menu table, and the grid is refreshed.	The new menu item was validated and saved to the database. A success message confirmed the number of rows inserted and the grid reloaded.	None
21	Edit Menu Item	Verify that a chef can edit any field of an	The edited row is updated in the Menu table after	Edits to menu item fields were saved to	None

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
		existing menu item directly in the grid and save the changes.	clicking Save. The MenuID column remains read-only to prevent accidental key changes.	the database correctly. The MenuID column was locked and could not be changed.	
22	Delete Menu Item	Ensure that a chef can select and delete one or more menu items, with a confirmation prompt before the deletion is committed.	After pressing Delete and confirming, the selected rows are permanently removed from the Menu table and the grid is refreshed.	Selected menu items were deleted from the database after confirmation. A success message indicated the number of rows removed.	None
23	View Orders	Verify that a chef can view all orders from the Order table in a read-only data grid.	All order records (OrderID, CustomerID, OrderDate, OrderStatus, OrderRemarks) are displayed. The grid is read-only.	All orders were loaded from the Order table and displayed in the read-only grid with correct data.	None
24	Update Order Status	Check that a chef can change the OrderStatus and OrderRemarks of an order directly in the editable grid and save.	The updated order status and remarks are saved to the Order table. OrderID, CustomerID, and OrderDate columns remain locked.	The order status and remarks were updated in the database. Locked columns could not be edited, and a success message was shown.	None

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
25	Update Chef Profile – Validation	Verify that the chef profile form validates: name required, email format (regex), phone 7–15 digits only, birthday not in the future, password minimum 8 characters.	Error messages are shown for empty name, invalid email, out-of-range or non-numeric phone, future birthday, and short password. Valid data is saved to the Chef table.	All validation rules were correctly enforced. Valid profile data was saved to the Chef table, and a success message was displayed.	None
Manager					
26	View Manager Dashboard	Verify that a manager's own profile data and the full feedback list are loaded when the Manager Home form opens.	The manager's profile grid shows only their own record (filtered by ManagerID). The feedback grid loads all feedback records from the Feedback table.	The manager's profile was retrieved and displayed correctly. All feedback records were loaded into the feedback grid on form load.	None
27	Update Manager Profile	Check that a manager can edit their own profile fields (name, email, phone, birthday, address, password) and save the changes.	After validation (email must contain '@' and '.', phone exactly 11 digits, birthday not in future), the Manager table is updated, and a success message is shown.	The manager profile was updated in the database after passing all validation checks. A success message confirmed the update.	None
28	View & Respond to Feedback	Verify that a manager can select a	The response text is validated (not empty, at least 2	The feedback response was saved to the	None

Test Case	Function Name	Test Objective	Expected Result	Actual Result	Remarks
		feedback record, read the customer text and order details, type a response, and save it.	characters, letters/digits/spaces only). The Feedback table is updated with the response text, ManagerID, and ResponseDate.	Feedback table after passing validation. A confirmation message was displayed and the form closed.	
29	Feedback Response Validation	Ensure that the feedback response form rejects an empty response, a response that is only one character, and a response containing special characters.	Error messages are shown for: empty response, response shorter than 2 characters, and response containing non-alphanumeric characters other than spaces.	All three validation rules were triggered correctly for invalid inputs. A valid response was accepted and saved.	None
30	View E-Wallet Report – Filter by Customer & Month	Verify that a manager can filter the e-wallet transaction report by selecting a specific customer and a year-month using the date picker.	The report grid shows Balance records matching the selected customer and period. Total Top-Up and Total Deduction labels display the correct aggregated amounts (cancelled orders are excluded from totals).	The report filtered correctly by customer and month. Totals were calculated accurately and cancelled order amounts were excluded from the deduction total.	None

5. Conclusion

The Gr8Food Management System was successfully developed and supports four user roles which includes System Admin, Manager, Chef, and Customer. Each roles have their own dedicated interface and functionalities.

The System Admin module provides management abilities allowing admins to add, update, and remove user accounts across all roles, as well as view and filter detailed sales reports based on month, year, food, category, and chef. The Manager module enables managers to view and respond to customer feedback and monitor e-wallet transaction reports filtered by customer and date period. The Chef module can be used by the chefs to manage the restaurant menu by adding, editing, and deleting items, updating menu availability status, and tracking as well as updating customer order statuses. Finally, the Customer module delivers a complete dining experience, allowing customers to top up their e-wallet, browse available menu items by category, place orders with automatic refunds, submit feedback for completed orders, and update their personal profile.

The system applies a clear execution of object oriented principles, with each database table represented by a dedicated class. The centralized DBHelper class ensures consistent database connection management across all role modules/ Comprehensive input validation is also implemented across all forms, preventing any invalid data entry. The role-based login system is logically connected across all roles. Top-ups, deductions and cancellations are automatically recorded in the Balance table and reflected in both the Manager's e-wallet report and the Admin's customer table view. When a Chef marks an order as Complete, the Customer gains the ability to submit feedback which the Manager can then respond to.

One notable thing that can be improved is that the e-wallet balance is stored as an integer, which limits the precision for long-term values. All users of the same role share a single User ID, reducing authentication security and scalability. The system currently lacks real=tome notifications, where Customers must manually check their order statuses, and Managers are not alerted when a new feedback is submitted. The Customer module does not display the Manager's responses regarding their submitted feedback. Additionally, the Admin's sales report only shows completed orders which limits visibility into cancelled or in-progress order trends.

The enhancements that the system could have in the future includes, assigning a unique UserID per individual user rather than sharing one ID per role which will improve security and traceability. The feedback system could be improved so customers can view Manager responses within their own interface, completing the two-way communication loop. Finally, the Chef module could include order notifications to allow chefs to respond to new customer orders more promptly.

6. References

1. Visual Paradigm. (n.d.). UML class diagram tutorial. Visual Paradigm Guides. Retrieved May 17, 2026, from https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/ (visual-paradigm.com in Bing)
2. Creately. (n.d.). What is a class diagram? Creately Guides. Retrieved May 17, 2026, from https://creately.com/guides/what-is-a-class-diagram/
3. Zhihu. (2020, June 30). How to write a good programmer's resume? Zhihu Zhuanlan. https://zhuanlan.zhihu.com/p/149035395
4. GeeksforGeeks. (2026, January 21). Use case diagram – Unified Modeling Language (UML). GeeksforGeeks. Retrieved May 17, 2026, from https://www.geeksforgeeks.org/system-design/use-case-diagram/
5. W3Schools. (n.d.). SQL tutorial. W3Schools. Retrieved May 17, 2026, from https://www.w3schools.com/sql/

7. Workload Matrix

No.	Assigned Task & Brief Description	Assigned Member Name	Status/ Comment
1.	1. Admin role page design and documentation; 2. Responsible for combining all files into the final integrated system 3. Helped fix bugs on other Role's code when combining 4. Helped Create the Manager Class Diagram 5. Fixed Update own profile form	Darren Matthew Tanoyo [TP086059]	Completed
2.	1. Team Leader	Duan Yuqing	Completed

	2. Manager pages design and coding; 3. Database creation and queries; 4. Prepare use case diagram for documentation 5. Helped fix bugs on other Role's code when combining 6. Explained the Manager class diagram 7. Helped with combining the files 8. Created Update own profile form	[TP114820]	
3.	1. Customer pages development; 2. Login pages development; 3. Responsible for writing the conclusion section of the report 4. Helped with admin dashboard	Farah Zakiya [TP092149]	Completed
4.	1. Chef pages design and coding; 2. Prepare and run system testing to validate functionality for test plan 3. Create and explained the Chef class diagram	Omar Ahmed Ramy Mohammed Elsayed Hassan Mansour [TP078735]	Completed